

Evaluating Suitability of the Actor Model of Concurrency for High-Frequency Trading

The `fast_send` Mechanism and a Tick-to-Book Latency Study

Vincent Maciejewski

M2 Technologies

mayeski@gmail.com

September 2026

Abstract

The many benefits of the actor model — per-actor state isolation, freedom from data races, and sequential single-message reasoning — have been recognized for decades, yet the model has long been dismissed as unsuitable for high-frequency trading (HFT). The reasoning was that actors imply many threads, many mailbox queues, and a message — with its heap allocation, queue operations, scheduler dispatch, and context switch — for every interaction: overhead incompatible with a microsecond latency budget. This paper argues that the dismissal is wrong for co-located actors, and supports the argument not only analytically but with a deployed, measured implementation — *kaspar-hft*, an open-source C++20 framework that extends the actor model into a form suitable for HFT and is its reference implementation. The extensions are four: `fast_send`, a synchronous delivery mechanism in which the sending thread runs the receiver’s handler inline and takes the reply as a value; actor *groups*, which co-schedule a set of actors on one thread behind a single shared mailbox; selectable mailbox queue implementations matched per actor to its write pattern; and a memory pool that removes per-message allocation and its tail.

The central mechanism is `fast_send`, whose defining property is *receiver transparency*: the handler is written identically for synchronous and asynchronous delivery and does not depend on which was used, which thread executes it, or whether the sender is blocked. Synchronous delivery across a chain of grouped actors runs entirely on the first actor’s thread, reducing scheduler context switches from $O(N)$ on the asynchronous path to $O(1)$; a thread-local call-chain test detects cyclic synchronous invocation on one thread before any lock is taken, while a cycle spread across threads is outside what it can see and deadlocks; and five delivery operations expose the latency/robustness spectrum behind one transparent interface. Microbenchmarks put the synchronous round trip at tens of nanoseconds — roughly two orders of magnitude below the ungrouped asynchronous path — and a single `fast_send` dispatch at about the cost of one unpredictable virtual call (about $5\times$ a direct call); the pool cuts the allocator tail by two orders of magnitude.

We then measure the framework in a real HFT setting: socket-to-book (“tick-to-book”) latency on a live CME market-data feed for ES, NQ, and ZN futures. The latency decomposes into a $\sim 7\ \mu\text{s}$ decode-and-book floor plus a per-message serialization slope, and the framework’s own per-message contribution is under 1% of that floor, which is measured with software timestamps from socket read to book update. The tail is set not by the actor machinery or the mailbox queue — which is convex, as queueing theory predicts, but fires on 0.08% of messages — but by the arrival process, which is non-Poisson and strongly clustered, consistent with a self-exciting process (under a Hawkes interpretation the observed Fano scaling implies a branching ratio $\approx 0.85\text{--}0.97$).

The latency the concurrency model adds is thus minor next to the irreducible latencies a real HFT data path already endures. Finally, the shared-queue group yields a production/simulation duality: the same actor code runs unchanged in live trading and in deterministic backtest. A saturated-throughput and contention evaluation is left to future work.

Keywords: actor model, message passing, concurrency, synchronous delivery, deadlock, receiver transparency, low-latency systems.

1 Introduction

In the actor model [1, 2], a program is a collection of isolated actors, each with private state, a message handler, and a mailbox. Actors communicate only by asynchronous message passing: a sender places a message on the receiver’s mailbox and proceeds without waiting, and the receiver dequeues and processes messages one at a time on its own thread. This discipline is what gives the model its guarantees — isolation, sequential per-actor processing, and location transparency — and it is essential when actors are distributed across machines.

When the communicating actors reside in the same process on the same machine, however, the asynchronous path carries overhead that the isolation guarantee does not require. A single request-and-reply between two co-located actors incurs, on conventional implementations:

- **Heap allocation.** Because the message must outlive the sender’s current continuation and persist until processed at an indeterminate later time, it must be heap-allocated, with the associated allocation cost, fragmentation, and garbage-collection pressure.
- **Queue operations.** Each message is enqueued by the sender and dequeued by the receiver, each requiring synchronization on the mailbox.
- **Scheduling and context switches.** The receiver must be scheduled onto a thread to run, incurring a context switch; a context switch typically costs on the order of 10^3 – 10^4 processor cycles and pollutes cache and translation lookaside buffer (TLB) state.
- **Round-trip latency.** A reply repeats the entire path in reverse, so a request-and-reply crosses the mailbox twice and the scheduler twice.

For a same-process pair, all of this is avoidable in principle: the sender’s own thread could run the receiver’s handler directly and take the reply as a return value. The difficulty is doing so without breaking the abstraction the actor model exists to provide. A direct handler invocation — calling the receiver’s handler as an ordinary method — discards the receiver’s exclusive-execution guarantee and exposes the receiver as a passive object. The design problem is to obtain the performance of a direct, same-thread call while preserving, from the receiver’s point of view, the semantics of message passing.

This paper describes **fast_send**, a synchronous delivery mechanism that resolves that tension. The sending thread acquires exclusive access to the receiving actor and executes the receiver’s handler directly, on the sender’s thread, returning the reply as a value. The receiver’s handler is identical for synchronous and asynchronous delivery and does not depend on which was used — the property we call *receiver transparency*. Because synchronous invocations can form cycles, and a cycle deadlocks a non-reentrant lock or overflows the stack under a reentrant one, the mechanism carries a lightweight cyclic-invocation detector that fires before any lock is taken.¹ It sees cycles on

¹kaspar-hft does not implement this detector yet: **fast_send** takes the receiver’s mutex directly, and its only check is a debug-build assertion against an actor synchronously sending to itself, so a synchronous cycle currently deadlocks.

one thread; a cycle spread across threads needs a different mechanism (Section 8). `fast_send` is implemented in *kaspar-hft*, an open-source C++20 actor framework for high-frequency trading;² all code and measurements in this paper are drawn from that repository.

Contributions.

1. A synchronous, same-thread actor-delivery mechanism (`fast_send`) that eliminates the heap allocation, queue operations, scheduling, and context switches of the asynchronous path for co-located actors, and returns the reply as a value (Section 6).
2. A precise statement of *receiver transparency*: the handler executes identical code in both modes and does not depend on the delivery mode, the executing thread, or whether the sender is blocked (Section 7). It is a property of how handlers are written, not one the framework or the language enforces.
3. A thread-local call-chain membership test that detects cyclic synchronous invocation on one thread before lock acquisition, together with a refined (actor, message-class) variant that admits callback patterns while still catching unbounded recursion (Section 8).
4. A family of five delivery operations spanning asynchronous through spin-wait synchronous to best-effort-with-async-fallback, unified behind one transparent handler interface (Section 9).
5. An analytical account of the per-message work eliminated, together with microbenchmarks of the C++ implementation that measure each eliminated operation and the pool’s tail benefit; a saturated-throughput evaluation is left to future work (Section 11).

2 Origins and history of the actor model

Invention (1973). The actor model was introduced by Carl Hewitt, Peter Bishop, and Richard Steiger in “A Universal Modular ACTOR Formalism for Artificial Intelligence,” presented at the Third International Joint Conference on Artificial Intelligence (IJCAI) at Stanford in August 1973 [1]. Its origins were in artificial-intelligence research at MIT: the formalism grew out of Hewitt’s earlier work on the PLANNER language and was proposed as a single, uniform notion — the *actor*, an entity that communicates only by sending messages to other actors — rich enough to serve as a foundation for AI programming. Hewitt drew on the object and message-sending ideas of Simula 67 and Smalltalk, on Lisp, and on capability systems, Petri nets, and packet-switched networks; in later retrospectives he described the model as inspired by the laws of physics rather than by any prior computational abstraction [22].

Formal foundations (1975–1986). The 1973 paper was informal, and a decade of work followed to give the model a rigorous semantics. Irene Greif’s 1975 MIT doctoral thesis supplied the first operational semantics for actor computations [18]; Hewitt and Baker stated “laws” characterizing the partial ordering of events in a distributed actor computation [19]; and William Clinger’s 1981 thesis gave a denotational semantics based on power domains, addressing the unbounded nondeterminism that distinguishes actors from other concurrency formalisms [20]. The foundation most cited today is Gul Agha’s doctoral work, published as *Actors: A Model of Concurrent Computation in Distributed Systems* [2], which recast the model with a transition-system operational semantics built on the

²<https://github.com/vincent212/kaspar-hft>

primitives *create*, *send*, and *become*, and moved the emphasis from artificial intelligence to a general theory of open, concurrent, distributed computation.

From theory to practice. The most influential early *deployed* system with actor-like structure — isolated processes with private state that communicate only by asynchronous message passing — was Erlang, developed at Ericsson from 1986 by Joe Armstrong, Robert Virding, and Mike Williams for fault-tolerant telephony [5]. The historical relationship to Hewitt’s model is often overstated: Erlang’s designers have said they were unaware of the actor model while building the language, and Armstrong later characterized Erlang’s concurrency as closer in spirit to communicating sequential processes than to the actor model [21]. Erlang is therefore best understood as a parallel, largely independent arrival at similar principles rather than a direct descendant, even though it is now routinely described as an actor language. Hewitt himself continued to revise the model’s definition well after 1973, emphasizing unbounded nondeterminism, locality, and guaranteed delivery in a much later retrospective [22]. The subsequent two decades produced a broad ecosystem of practical actor runtimes — on the JVM, the .NET platform, C++, Rust, Swift, and elsewhere — which we survey next.

3 A survey of actor-model implementations

This section surveys the major actor runtimes along the three axes most relevant to **fast_send**: how actors are mapped to operating-system threads, whether and how a synchronous or request–reply path is offered, and what is known of their performance. The consistent findings across the ecosystem — that actors are multiplexed M:N over a small worker pool, and that no mainstream system runs the *receiver’s* handler on the *sender’s* thread — are what locate the contribution of **fast_send**. Table 1 summarizes the comparison; the per-system detail and performance figures follow. The later related-work discussion (Section 5) revisits the systems closest to **fast_send** to sharpen the distinction; here the account is descriptive.

BEAM: Erlang and Elixir. The Erlang runtime multiplexes very large numbers of lightweight processes — each with a private heap and mailbox — onto a small set of scheduler threads, one per logical core by default, with a per-scheduler run queue and work-stealing migration [23]. Scheduling is preemptive by a reduction count: a process runs until it consumes its budget (4000 reductions by default) and is then descheduled, which bounds tail latency [23]. The synchronous path, `gen_server:call`, blocks the caller for up to a timeout and returns a value, but the request is placed in the *server* process’s mailbox and handled by `handle_call/3` on the server’s own scheduler; the caller merely performs a selective receive on the reply. The caller’s thread never runs the callee’s handler [5].

JVM: Akka. Akka does not bind actors to threads. A dispatcher binds a set of actors to a fork–join thread pool, and an actor occupies a thread only while it has messages to process [7]. A `throughput` setting caps how many messages an actor processes before the thread moves to the next actor; this mailbox batching is the principal latency/throughput lever. Akka’s own microbenchmark reports on the order of 50 million messages per second on a single machine in a ping-pong configuration, a figure the authors show is dominated by that batching setting; it is a vendor microbenchmark, not an application measurement, and should be read as such [24]. The request–reply operator `ask` returns a `Future/CompletionStage` rather than a value and never

Table 1. Actor-to-thread mapping and synchronous messaging across implementations. The final column — whether the sender’s thread ever executes the receiver’s message handler — is negative for every surveyed system and positive only for **fast_send**.

System	Runtime	Actor → thread mapping	Synchronous request-reply	/ Sender runs handler?
Erlang/OTP, Elixir	BEAM VM	M:N green processes; one scheduler thread per core; per-scheduler run queues, work stealing	<code>gen_server:call</code> blocks caller, returns a value; handler runs in the server process	No
Akka	JVM	thread-pool dispatchers over a fork-join pool; actors not 1:1 with threads	<code>ask (?)</code> returns a <code>Future</code> ; <code>tell (!)</code> is fire-and-forget	No
Akka.NET, Orleans	.NET	thread-pool / turn-based scheduling; grains single-threaded per activation	grain methods return <code>Task<T></code> ; always asynchronous	No
CAF	C++	cooperative work-stealing scheduler over $P=\#\text{cores}$ workers; <code>detached</code> actors get a thread	<code>request</code> returns a response handle; <code>.then</code> continuation, non-blocking	No
SObjectizer	C++	agents bound to dispatchers supplying worker threads/pools	async by default; <code>so5extra::request_reply</code> blocks; runs on provider’s dispatcher	No
Actix	Rust / Tokio	actors are tasks on Arbiter event-loop threads (M:N)	<code>send</code> returns a <code>Request</code> future; <code>do_send</code> fire-and-forget	No
Pony	Pony RT	work-stealing scheduler, one thread per core; per-actor heaps	none — behaviours are asynchronous by design	No
Swift actors	Swift RT	serial executor per actor over a cooperative pool	cross-actor access requires <code>await</code> ; synchronous cross-actor calls are prohibited	No
Ray, Dapr, distributed Proto.Actor		one worker/process or mailbox per actor; M:N pools	remote call returns a future; turn-based dispatch	No
fast_send (this work)	C++/Rust	per-actor exclusive-access lock; sender’s thread runs the handler	blocking call returns a value on the sender’s thread	Yes

blocks the caller on the handler; Akka’s documentation explicitly discourages blocking inside actors because it starves the shared pool [7].

.NET: Orleans and Akka.NET. Orleans introduced the “virtual actor” (grain) abstraction, in which the runtime activates grains on demand and schedules them turn-based and single-threaded over a shared pool; every grain method is asynchronous and returns a `Task`, so a caller obtains a promise and the grain body runs on the grain’s own activation context, never on the caller’s thread

[9]. The Orleans team reported the Halo 4 presence service sustaining roughly 130,000 requests per second across 25 servers with near-linear scaling to 125 servers [25]. A recent independent empirical comparison of Akka, Akka.NET, and Orleans on commodity cloud hardware found Akka.NET and Orleans comparable and both ahead of Akka in a non-distributed setup, with Orleans strongest in the distributed setup [27].

C++: CAF and SObjectizer. The C++ Actor Framework schedules cooperatively-run actors over a work-stealing pool of one worker per core, with a lock-free mailbox; actors that must block opt into a `detached` mode that gives them a dedicated thread [8]. Its request-reply operation, `request`, does not occupy a CAF thread while waiting: the requesting actor simply processes no other message until the response arrives, and installs a typed continuation via `.then`; the caller does not execute the callee’s handler [8]. On the Savina cross-language benchmarks, a recent study found CAF only marginally ahead of Akka on average (about $1.09\times$ at 20 threads) and behind it on nine of sixteen benchmarks [16], so the common claim that native C++ actors dominate the JVM is more nuanced than it is often stated. SObjectizer likewise binds agents to dispatchers backed by worker-thread pools and is asynchronous by default; its companion library offers a synchronous `request_reply`, but the handler still runs on the service provider’s dispatcher, not the caller’s thread.

Rust: Actix. Actix runs actors as tasks on Arbiter event loops atop the Tokio runtime, multiplexing many actors per thread. `send` returns a `Request` future that resolves to the handler’s result, and `do_send` is fire-and-forget; in both cases the handler runs on the target actor’s Arbiter, not the sender’s. (The widely-cited Actix TechEmpower web-framework results concern `actix-web`, not the actor library, and are not evidence about actor-message performance.)

Pony. Pony is the clearest case of the mainstream design point. Behaviours are asynchronous by construction — there is no synchronous send at all — so request-reply must be built by hand from callback messages. Pony runs a work-stealing scheduler with one thread per core and gives each actor its own heap, which enables a fully concurrent, per-actor garbage collector (ORCA) that needs neither read nor write barriers, relying instead on reference capabilities for data-race freedom and on causal message delivery for correctness [26]. Its performance comes from cheap actors, local GC, and never blocking.

Swift. Swift’s language-level actors (SE-0306) give each actor a serial executor drawn from a cooperative thread pool. Cross-actor access must be `await`-ed and is therefore asynchronous; the compiler *prohibits* synchronous cross-actor calls outright [10]. Swift actors are reentrant: a suspended actor may interleave other work before resuming.

Distributed runtimes. Ray, Dapr, and Proto.Actor extend the actor idea across machines. A Ray actor is by default a single worker executing methods one at a time, and a call returns an object reference (a future); Dapr enforces turn-based, single-threaded access to each actor via a per-actor lock and dispatches method calls over HTTP/gRPC; Proto.Actor processes each mailbox one message at a time on a shared thread-pool dispatcher. In all three the invocation is asynchronous and the sender never runs the receiver’s handler.

Two structural facts. Two observations hold across the entire survey. First, on *thread mapping*: the dominant design multiplexes many actors M:N over a small pool of worker or scheduler threads,

because assigning a dedicated operating-system thread to each actor does not scale to large actor populations; the few one-thread-per-actor cases (CAF’s **detached** actors, Ray’s threaded actors, process-per-actor deployments) are reserved for blocking or isolation needs and are understood as exceptions. Second, on *synchronous messaging*: where a synchronous or request–reply path exists at all, it is emulated over asynchronous mailbox delivery — the caller either blocks in a receive (Erlang) or, more commonly, receives a future/promise (Akka, Orleans, CAF, Actix, Ray) — and in every case the receiver’s handler runs on the receiver’s own scheduled thread or context. Some systems (Pony, Swift’s cross-actor calls) forbid a synchronous call altogether. No mainstream actor runtime executes the receiver’s handler on the sender’s thread under the receiver’s exclusive-access lock. That combination — cross-thread synchronous execution under receiver transparency — is the design point **fast_send** occupies. It needs protection against synchronous cycles, which Section 8 provides for cycles on one thread.

4 Why actors? The design case and its cost

A fair question precedes any discussion of making actors faster: why adopt the actor model at all, when a program can already spawn operating-system threads and share memory under mutual exclusion, or drive concurrency through an event loop such as `boost::asio`? Those mechanisms are not deficient in capability. The case for actors is not that they compute something the alternatives cannot; it is what the discipline *removes*, and the classes of error it makes unrepresentable.

What the discipline removes. Because each actor owns its state and communicates only by messages, there is no mutable state shared between actors, and therefore no memory-level data race — the defining and most pernicious failure of lock-based concurrency. Empirical study of the field bears this out: data races and deadlocks are described as “two mistakes that are hard to make with actors” [32], and a comprehensive study of 186 real-world actor bugs confirms that actor programs are markedly less susceptible to low-level data races and deadlocks than multithreaded code, even as they admit their own higher-level bug classes [33, 34]. The programmer reasons about one message at a time against a consistent private state, sequentially, and never writes a lock. The synchronization apparatus that dominates the correctness burden of hand-rolled concurrent code — choosing lock granularity, establishing a global lock order to avoid deadlock, placing memory barriers, and reasoning about every interleaving that a thread scheduler or an `asio` callback chain admits — simply does not appear in an actor program. Fault isolation follows from the same isolation: a failing actor can be restarted by a supervisor without corrupting the state of others, the foundation of the “let it crash” reliability discipline [6].

These properties belong to the asynchronous model. The synchronous delivery of Section 6 gives one of them up: a **fast_send** holds the receiver’s lock while its handler runs, and a chain of **fast_send** calls holds several actors’ locks at once. A cycle of synchronous calls on one thread is caught by the detector of Section 8; a cycle across threads, where thread 1 holds *A* and waits for *B* while thread 2 holds *B* and waits for *A*, deadlocks, exactly as a lock-ordering error does in lock-based code. The asynchronous path keeps the guarantee; the synchronous path trades part of it for latency.

The cost. These properties are not free. The message-passing discipline that buys isolation is exactly the source of the per-message overhead this paper is concerned with: heap allocation, mailbox queue operations, scheduling, and context switches (Section 11). This is the tax paid for isolation, and on the hottest paths it has historically been the reason engineers abandon the actor

model and return to shared memory and locks. The tension between the model’s safety and its per-message cost is the subject of this paper.

Determinism and simulation. A further benefit follows from the same isolation and becomes decisive in a trading system. Because an actor’s behavior is a function of the message sequence on its mailbox and nothing else — no shared state, and no wall-clock or global-random side channel once the actor obtains time and randomness as messages — an actor is deterministically reproducible: the same input sequence yields the same states, outputs, and faults on every run. This makes the actor a natural unit of *simulation*. When actors are co-scheduled behind one queue (Section 11.1), that single queue is fed by live sources in production or replayed from a recorded or synthetic sequence in simulation, and the actors cannot tell the two apart. The same code is therefore both a real-time execution engine and a deterministic simulator: a trading strategy is developed and backtested against recorded market data, debugged by replaying the exact sequence that produced a fault, and deployed to production without modification. Shared-memory and callback-based designs do not offer this without effort — their behavior depends on thread interleavings and ambient clocks that a simulator must otherwise mock away, whereas for actors the property falls out of the message discipline. The duality, and its use in developing an execution algorithm, are the subject of Section 13.

The generation argument. There is a further benefit that is only now coming into view, and it may in the end be the most consequential. An actor is a small, regular unit of code: a class with private state and one handler per message type, whose entire observable contract is the set of messages it accepts and the replies it returns. This regularity makes actors unusually amenable to automatic construction by large language model (LLM) code generators, which are fluent in the paradigm: a model can scaffold an actor, write its message handlers, and — because an actor’s behavior is a function from an incoming message to a reply with no hidden shared state — generate self-contained unit tests that drive messages in and assert on the replies, without the elaborate harness that testing shared-memory code requires. The contrast with the alternative is sharp and is documented. Generating *correct* lock-based concurrent code is hard even for strong models: a dedicated benchmark for concurrent code generation finds that current LLMs struggle to produce correct synchronization and repeatedly introduce concurrency errors [35]. The actor discipline eliminates the hazard that both human authors and code generators handle worst — shared mutable state guarded by hand-written locks — so actor code is not only faster to produce but is likely to be more robust and closer to correct than a hand-rolled alternative built on a custom locking scheme. (We state this as a design argument; the load-bearing evidence is the demonstrated difficulty of the lock-based alternative [35], together with the measured data-race freedom of the actor approach [32, 33].)

Removing the cost makes the benefits free. Taken together, the benefits of the actor model — data-race freedom, sequential per-actor reasoning, fault isolation, and now amenability to reliable automatic generation and testing — have always been offset by a performance penalty that put the model out of reach on latency-critical paths. If that penalty can be removed for the common case of co-located actors, the calculus changes: the isolation, the absence of hand-written locking, and the generation-friendliness are retained, while the per-message cost approaches that of a direct call. The benefits, in other words, accrue for free. Providing that path, safely and without breaking the abstraction, is what `fast_send` is for, and it is the mechanism to which we now turn.

5 Background and related work

The actor model [1, 2] specifies asynchronous message passing as the sole means of inter-actor communication, and foundational and subsequent treatments present this asynchrony as essential to isolation, fault tolerance, and location transparency [2, 15]. Several systems provide a second, synchronous-looking communication path; each differs from `fast_send` in a way that bears on receiver transparency or on multi-threaded execution.

Single-threaded synchronous calls. The E language [3] offers an immediate call `(.)` returning a value and an eventual send `(<-)` returning a promise. E vats are single-threaded, so the immediate call is a conventional method invocation with no lock, no cross-thread execution, and no possibility of deadlock; the model does not extend to actors running on multiple threads. AmbientTalk [4] similarly builds on communicating event loops in which cross-actor communication is always asynchronous. Actor4j [11] optimizes messaging between actors on the *same* thread, and reliable-actor systems such as the Kubernetes Application Runtime (KAR) [12] bypass the queue only for an actor’s calls to itself, not for cross-actor calls.

Blocking that still uses the receiver’s thread. Erlang’s `gen_server:call` [5] blocks the caller, but the request still traverses the receiver’s mailbox and is processed on the receiver’s own process; the caller’s thread never executes the receiver’s handler. Akka’s `ask` operator [7] returns a future and remains asynchronous internally, routing through the mailbox, and the framework explicitly discourages blocking inside actors. Orleans [9] presents grain methods in call syntax but is asynchronous underneath (Task-returning), with no caller-thread execution of the grain. Swift’s actors [10] require `await` for cross-actor access and prohibit synchronous cross-actor calls at the compiler level. The C++ Actor Framework (CAF) [8] dispatches handlers on a scheduler’s worker pool, so the caller thread does not run the target’s handler.

Direct execution without message-passing semantics. The Active Object pattern [13] interposes a proxy, activation queue, and scheduler so that the client’s thread never runs the servant’s method on a separate thread. The Monitor Object pattern [14] does run a method on the caller’s thread under a lock — sharing the caller-thread-execution mechanism with `fast_send` — but a monitor is a passive object invoked by a direct method call: it has no mailbox, offers no asynchronous alternative, and does not present the callee as an isolated actor that cannot observe how it was reached.

Deterministic reactor concurrency. A separate recent line of work constrains, rather than accelerates, actor-style concurrency: Lingua Franca [16] schedules reactors under a logical-time model to make concurrent execution deterministic. That objective is orthogonal to `fast_send`, which addresses the per-message cost of co-located delivery and preserves receiver transparency; the two could be combined, since `fast_send` does not alter the logical ordering of message handling.

5.1 Positioning

Across these systems, a synchronous path either (i) is confined to a single thread or same-thread actors (E, AmbientTalk, Actor4j), (ii) blocks the caller while still executing on the receiver’s thread through the mailbox (Erlang, Akka, Orleans, CAF), (iii) is prohibited outright (Swift), or (iv) abandons message-passing semantics for direct method calls on a passive object (Active Object, Monitor Object). What distinguishes `fast_send` is the combination of (a) cross-thread synchronous

execution — the sender’s thread runs the receiver’s handler while holding the receiver’s exclusive-access lock — (b) under receiver transparency, with (c) a cyclic-invocation detector that catches synchronous cycles on one thread before they deadlock or overflow the stack; cycles across threads are outside what it can see (Section 8). The remainder of the paper develops these three elements.

6 The synchronous delivery mechanism

Each actor has a message handler, a mailbox, private state, and a synchronization mechanism that guarantees at most one thread executes its handler at a time (a mutex by default; a lock-free primitive is admissible). Two delivery operations share this structure.

Asynchronous delivery (`send`). The sender heap-allocates the message, enqueues it on the receiver’s mailbox under the mailbox’s synchronization, and returns immediately with no value. Later, a thread assigned to the receiver dequeues the message, acquires the receiver’s exclusive-access lock, and runs the handler. A reply, if any, is heap-allocated, wrapped in a response message, and enqueued on the sender’s mailbox.

Synchronous delivery (`fast_send`). The sender may allocate the message on its *stack*, because the sending thread blocks until processing completes and the message’s lifetime is therefore bounded by the call. The sender’s thread acquires the *receiver’s* exclusive-access lock (blocking if another thread holds it), pushes the receiver onto a thread-local call chain (Section 8), and invokes the receiver’s handler directly on the sender’s thread. The handler runs its identical code and optionally calls the reply operation; on return, the receiver is popped from the call chain, the lock is released, and `fast_send` returns the reply value (or null if none was set). A stack-allocated message is reclaimed when the sender’s frame unwinds.

In both modes the receiver’s exclusive-access lock is held for the duration of handler execution, so the “one message at a time” guarantee holds identically under the conservative detection policy; the permissive (actor, message-class) refinement of Section 8 admits same-actor re-entrancy and relaxes it. The distinction from a naive direct call is that the synchronous path holds the receiver’s exclusive-access lock and routes the reply through the same operation as the asynchronous path; it is delivery of a message that happens to execute on the sender’s thread.

Unified reply routing. A single reply operation serves both modes and selects its behavior from delivery-context information rather than from any argument the handler supplies. Under asynchronous delivery it heap-allocates the value, wraps it in a response message, and enqueues it on the sender’s mailbox; under synchronous delivery it heap-allocates the value and stores it in a return-value location that the blocked `fast_send` reads and returns. The value is heap-allocated in *both* modes deliberately: if the synchronous path could stack-allocate the reply while the asynchronous path could not, the handler would have to know the mode, defeating transparency. The reply may be invoked at most once per message.³ An opt-in optimized reply stores the value without heap allocation for code paths known to be synchronous-only;⁴ it raises an error if the message was delivered asynchronously, and by design breaks transparency, so its use is explicit.

³kaspar-hft does not enforce this yet: a second reply to a synchronous delivery replaces the first, and a second reply to an asynchronous one sends a second message.

⁴kaspar-hft does not implement the optimized reply yet.

Chained synchronous execution. If actor A synchronously invokes B , which synchronously invokes C , and so on, the entire chain executes on A 's thread, which holds the exclusive-access locks of all actors in the chain simultaneously and, when those locks are uncontended, performs no context switches. Such an N -actor synchronous chain incurs $O(1)$ context switches, against $O(N)$ for the equivalent asynchronous chain, and executes on a single core, avoiding the cross-core cache-line migration of state that the asynchronous path incurs. A contended lock in the chain blocks its thread, which reintroduces a context switch at that step.

7 Receiver transparency

Receiver transparency is the property that the receiving actor's handler is written identically for both delivery modes and does not depend on (i) whether it was reached synchronously or asynchronously, (ii) which thread is executing it, or (iii) whether the sender is blocked awaiting a reply. It is a property of how handlers are written, not isolation that the framework or the language enforces. A handler that looks can infer the mode: C++ cannot stop it from inspecting its own execution context — its thread identifier, the address of a local variable (and so which thread's stack it is on), or the time — and some message fields the framework sets are filled differently in the two modes (element 4). Transparency means that a handler has no reason to look, and that its code does not change between modes. It is what keeps `fast_send` within the actor abstraction rather than reducing it to a locked method call.

Four elements bear on it:

1. **Identical interface.** The handler receives a message and invokes the same reply operation in both modes; there is no mode flag, and no conditional branch in the handler depends on the delivery mode.
2. **Exclusive access in both modes.** The receiver's exclusive-access lock is held throughout handler execution whether the handler runs on the receiver's own thread (asynchronous) or on the sender's thread (synchronous), so the observable execution environment — single-writer, sequential — is the same.
3. **Mode-independent reply and allocation.** The unified reply routes by delivery context with no handler involvement, and the reply value is heap-allocated in both modes, so neither the reply's behavior nor allocation side effects reveal the mode.
4. **No delivery flag for handlers.** The flag recording how a message was delivered is private to the framework. Other message fields are not mode-neutral: the queue depth and the “mailbox empty” flag describe the receiver's mailbox on asynchronous delivery and have fixed values on synchronous delivery, and the destination field is set only on asynchronous delivery. They exist for instrumentation and routing; a handler whose logic depends on them gives up transparency.

Two consequences follow. First, the delivery mode becomes a *sender-side* optimization: a sender may upgrade an asynchronous send to a synchronous one with no change, recompilation, or redeployment of the receiver. Second, the mechanism is distinct from a direct procedure call, in which the callee is a passive object invoked on the caller's stack with no mailbox and no exclusive-execution guarantee; here the receiver remains an isolated actor that perceives the arrival of a message and is unaware that the sender's thread is the one running it.

8 Cyclic-invocation detection

What the detector cannot see: cycles across threads. The call chain below is per thread, so it detects only a cycle that stays on one thread. It cannot see a cycle spread across threads: thread 1 runs a `fast_send` to *A* whose handler does a `fast_send` to *B*, while thread 2 runs a `fast_send` to *B* whose handler does a `fast_send` to *A*. Each thread’s check passes — $B \notin \{A\}$ and $A \notin \{B\}$ — and each thread then blocks on the lock the other holds. `fast_send` is callable from any thread, so this case is real. Catching it needs shared state rather than a thread-local chain: a global wait-for graph checked before blocking, a global order on actor locks, or a bounded try-lock that falls back to asynchronous delivery (the `fast_send_void` policy of Section 9). Choosing among these is future work (Section 15).

Synchronous invocation introduces a failure mode that asynchronous delivery does not have. If synchronous calls form a cycle — $A \rightarrow B \rightarrow C \rightarrow A$ — then under a non-reentrant lock the thread waits to acquire an actor’s lock it already holds, and the system deadlocks; under a reentrant lock the handler re-enters with the lock’s recursion count incremented, producing unbounded nested execution that overflows the stack and exposes inconsistent intermediate state. Neither is acceptable, and the cycle cannot be discovered after the offending lock is taken — by then the thread is already stuck.

8.1 Call-chain membership test

The mechanism maintains, per thread, a *call chain*⁵: the ordered set of actors whose handlers are currently executing on that thread via synchronous invocation. Because it is thread-local, it requires no synchronization and resets automatically between tasks. It is initialized when a thread begins processing a dequeued (asynchronous) message — the dequeued actor is its first entry — and each synchronous invocation pushes its target on entry and pops it on completion. The chain may be represented as a stack or list, a hash set for $O(1)$ membership queries, a bitmap indexed by actor identifier, or a hybrid.

Before acquiring the target’s lock, a synchronous send queries whether the target is already present in the chain. If it is not, the send proceeds normally; if it is, a cyclic invocation has been detected, and the mechanism signals it — by raising a cyclic-invocation exception carrying the target identity and the full chain, or by returning an error indicator — *without* acquiring the lock or invoking the handler. Algorithm 1 states the check. Because the query precedes lock acquisition and reads only thread-local state, its overhead is negligible against the cost of a lock acquisition and a handler invocation, and the chain depth is small in practice.

8.2 Refined detection with message classes

Membership keyed on actor identity alone is conservative: it rejects legitimate callbacks in which *A* synchronously invokes *B* and *B* synchronously invokes a *different* handler of *A*. A refined mode⁶ keys the call chain on (actor, message-class) pairs, where the message-class discriminator may be a type tag, a class identifier, a handler function pointer, or any code-path discriminator. A cycle is signaled only when the same pair recurs. This admits a callback pattern such as $A/\text{compute} \rightarrow B/\text{validate} \rightarrow A/\text{validated}$ — three distinct pairs — while still catching genuine unbounded recursion such as $A/\text{foo} \rightarrow B/\text{bar} \rightarrow A/\text{foo}$, in which a pair repeats. Because permissive keying allows an actor’s state to be re-entered by a nested handler, reentrancy safety in that mode is

⁵kaspar-hft does not implement the call chain yet.

⁶kaspar-hft does not implement the refined mode yet.

Algorithm 1: Synchronous send with cyclic-invocation detection

thread-local: call chain C (set of actors executing synchronously on this thread)

Input: target actor B , message m

```
1 if  $B \in C$  then
2   return cyclic-invocation signal (exception or error indicator) // before any lock is
   taken
3 acquire  $B$ 's exclusive-access lock
4  $C \leftarrow C \cup \{B\}$ 
5  $r \leftarrow B.\text{handler}(m)$  // runs on this (the sender's) thread
6  $C \leftarrow C \setminus \{B\}$ 
7 release  $B$ 's exclusive-access lock
8 return  $r$ 
```

the programmer's responsibility. The policy — conservative (actor-only) or permissive (pair-based) — is configurable system-wide, per-actor, or per-invocation.

9 A family of delivery operations

The same transparent handler interface supports five delivery operations⁷ that differ only in how the sender behaves when the receiver's lock is contended or a cycle is detected (Table 2). All synchronous variants perform the cyclic-invocation check of Section 8 before attempting the lock.

Table 2. The five delivery operations. All share one receiver-transparent handler interface; they differ in sender-side behavior under contention and cycles.

Operation	Blocks	Returns	Lock unavailable	Cycle detected
<code>send</code> (async)	no	—	(enqueues)	—
<code>fast_send</code> (blocking sync)	yes	value	waits (sleep on lock)	exception
<code>fast_send_spin</code>	yes	value	waits (busy-wait)	exception
<code>fast_send_x</code>	no	value	<code>ActorBusy</code> exception	exception
<code>fast_send_void</code>	no	—	async fallback	async fallback

- **fast_send** blocks on the receiver's lock by sleeping, and raises a cyclic-invocation exception on a cycle. It is the default synchronous operation.
- **fast_send_spin** is identical except that it busy-waits on the lock, optionally issuing a central processing unit (CPU) pause instruction between attempts, trading CPU cycles for lower wake latency by avoiding the scheduler. It suits short critical sections on latency-sensitive paths with spare cores.
- **fast_send_x** attempts the lock without blocking: on success it runs the handler and returns the value; on contention it raises an `ActorBusy` exception, distinct from the cyclic-invocation exception, so the caller can fall back, retry, or propagate. Its execution time is bounded.

⁷kaspar-hft implements `send` and `fast_send`; it does not implement the other three, or the adaptive variant, yet.

- **fast_send_void** attempts the lock without blocking and returns no value: on success it runs the handler; on contention *or* on a detected cycle it enqueues the message asynchronously and returns, copying a stack-allocated message to the heap first if necessary. Because it never blocks on the target lock, the operation itself cannot be the waiting party in a lock-wait deadlock; the handler it runs on success may still block or recurse. It fits notification, forwarding, pipeline, and broadcast patterns. The ratio of synchronous completions to asynchronous fallbacks is a useful contention metric.
- An adaptive variant spins for a bounded count and then transitions to blocking acquisition, combining low latency under light contention with bounded CPU use under heavy contention.

Selection among the operations is a sender-side decision governed by whether the caller needs a return value, can tolerate an unbounded wait, and has idle processor capacity available; the receiver’s handler is unaffected in every case.

10 Where latency comes from: queueing, not context switching

On a latency-critical message-handling path — of which a high-frequency trading (HFT) system is the archetype — the quantity that matters is not the mean latency but the tail: the 99th, 99.9th, and 99.99th percentiles. It is a common misconception that this tail is dominated by the cost of moving a message from one thread to another. It is not. On a modern Linux server a thread-to-thread context switch costs on the order of one to a few microseconds, and that cost is roughly fixed per hop: direct measurements put it near 1.2–1.5 μs when threads are pinned to a core, rising to a few microseconds once cache-working-set effects are included [37, 39, 38]. The tail is instead produced by *queueing*: when the code that services a queue cannot keep pace with the arrival rate, the queue grows, and the time a message spends waiting grows without bound. This is a structural property of queues, independent of any particular send primitive.

10.1 The queueing argument

Model one servicing stage as an M/M/1 queue [36]: messages arrive at rate λ and are serviced at rate μ , giving utilization $\rho = \lambda/\mu$; the queue is stable only for $\rho < 1$. The mean number of messages in the system is, by Little’s law,

$$L = \frac{\rho}{1 - \rho}, \quad (1)$$

and the mean sojourn time (waiting plus service) is

$$W = \frac{1/\mu}{1 - \rho} = \frac{1}{\mu - \lambda}. \quad (2)$$

As $\rho \rightarrow 1$ — as the servicing code approaches saturation — $W \rightarrow \infty$. The tail is governed by the same factor: in M/M/1 the sojourn time is exponentially distributed, $\Pr[T > t] = e^{-(\mu - \lambda)t}$, so its q -quantile is

$$t_q = \frac{\ln(1/(1 - q))}{\mu - \lambda} = \frac{\ln(1/(1 - q))}{\mu(1 - \rho)}. \quad (3)$$

The 99.9th percentile is therefore $t_{0.999} \approx 6.9/(\mu(1 - \rho))$: *every* quantile scales as $1/(1 - \rho)$, and the high percentiles are fixed large multiples of the mean. Figure 1 plots this. At $\rho = 0.5$ the 99.9th-percentile latency is about 14 service times; at $\rho = 0.9$ it is 69; at $\rho = 0.95$ it is 138. The knee is sharp, and the region past it is where HFT tails are born. Concretely, a queue whose depth

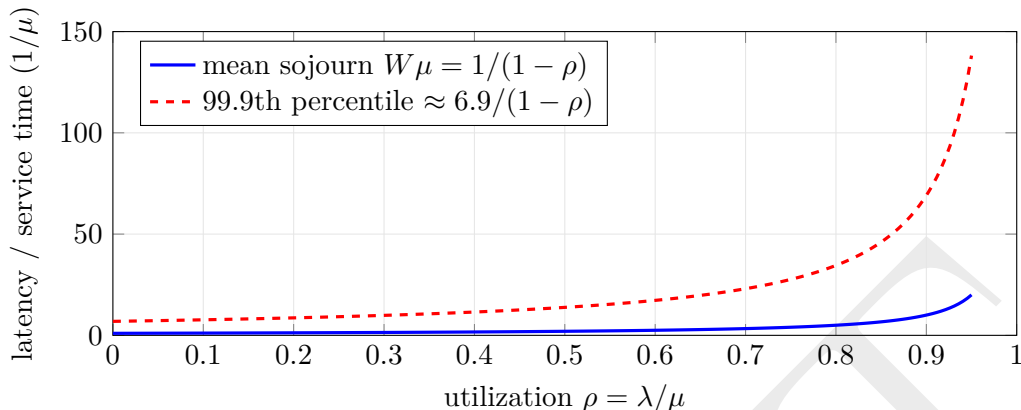


Figure 1. Latency versus utilization for an M/M/1 servicing stage, in units of the mean service time $1/\mu$. Both the mean and every tail quantile diverge as $\rho \rightarrow 1$; the tail is a fixed multiple of the mean but far larger in absolute terms. Keeping the queue short means keeping ρ well left of the knee, which means making the servicing code fast enough to raise μ .

hovers at 0 or 1 adds negligible latency, whereas one that backs up to hundreds or thousands of messages produces tails orders of magnitude above the mean.

The design consequence is direct. The only free parameters are λ and μ . The arrival rate λ — the market’s message rate — is not ours to set. The service rate μ is: it is the reciprocal of the time the servicing code takes per message. Halving the per-message service time doubles μ , which for a fixed λ moves ρ from, say, 0.9 to 0.45 and collapses the tail. This is why the aggregate speed of the queue-servicing code — not the latency of any single send — governs the tail. A single `fast_send` that saves tens of nanoseconds is uninteresting in isolation; it is decisive because, applied across every hop of the servicing chain, it raises μ enough to hold ρ far from 1 and the queue near empty. `fast_send` attacks the tail by attacking μ .

10.2 A representative HFT pipeline

Figure 2 shows the thread architecture of a representative HFT market-data-to-order pipeline, drawn from the `kaspar-hft` reference implementation. CME’s MDP 3.0 market data arrives as two redundant multicast feeds, channel A and channel B. The system decomposes into three sections connected by exactly two cross-thread hops.

Section 1 — capture (two threads). One socket-reader actor per channel does nothing but drain its UDP socket and hand each datagram to the next stage by an *asynchronous send*. It must be asynchronous: a reader that made a synchronous, blocking call would stop draining its socket while the callee ran and would drop packets under a burst — defeating the reader’s only purpose. In `kaspar-hft` the reader pre-allocates message buffers and batches whatever the socket has available before sending, and its handoff to the buffer stage is an `async send`, never a `fast_send`.

Section 2 — decode, book, and signal (one thread). The datagrams land in a buffer actor — the queue — whose service handler forwards each message down the remainder of the chain by `fast_send`: arbitration between the A and B feeds, Simple Binary Encoding decode, order-book construction, and signal generation all run synchronously, in sequence, on this one thread, with no intervening queue and no context switch between stages. This is the chained synchronous execution

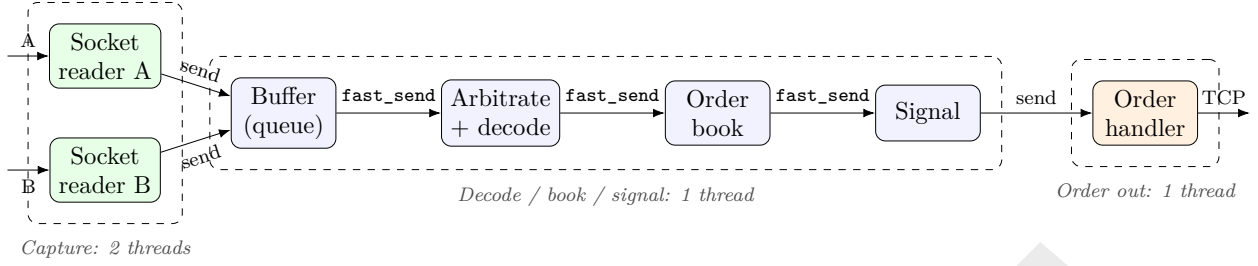


Figure 2. Thread architecture of a representative HFT pipeline. The only cross-thread hops — and hence the only context switches — on the critical path are the two asynchronous `send`s (reader→buffer and signal→order-handler); each such hop is the expensive operation, measured at $\sim 3.4\mu\text{s}$ per round trip (Table 4), because it wakes another thread. The entire decode/book/signal servicing chain instead collapses onto a single thread via `fast_send`, each hop costing $\sim 10\text{ ns}$ — about one virtual call, three orders of magnitude less than a thread-crossing `send` (see Table 4) — so it incurs no queue and no context switch. That servicing chain is the M/M/1 stage of Section 10.1 whose per-message service time, and hence μ , is set by the aggregate cost of these `fast_send`s.

of Section 6: an N -stage chain with $O(1)$ rather than $O(N)$ context switches. This section is the queue-servicing code whose per-message time sets μ , and it is where `fast_send` earns its keep.

Section 3 — order out (one thread). When the signal decides to act, it hands an order to an order-management actor by an asynchronous `send`; that actor performs the TCP/iLink write. This hop is asynchronous by design: the network write takes several microseconds, and blocking the decode-and-book thread on it would stall servicing of the market-data queue — driving ρ back toward 1, the exact failure the architecture exists to avoid.

Two hops, two switches. On the critical path from wire to order there are just two cross-thread hops — reader→buffer and signal→order-out — and therefore two context switches, a fixed cost of a few microseconds in total [37]. Everything between them is collapsed onto a single thread by `fast_send`. The goal is not to remove those two switches; they are cheap and necessary, because the capture and transmit stages must not block the servicing stage. The goal is to make the servicing stage between them fast enough that its queue never grows — and that is a statement about μ , which is what `fast_send` raises. The mechanism’s contribution is thus best measured not as a per-call latency but as the service rate it sustains on the servicing chain.

11 Performance analysis

The synchronous path removes, rather than optimizes, the per-message operations of the asynchronous path. Table 3 summarizes the operations eliminated for a co-located delivery.

Each eliminated item has a concrete cost. A context switch is on the order of 10^3 – 10^4 processor cycles and evicts cache and TLB state. Heap allocation of short-lived messages produces fragmentation and, in managed runtimes, collection pauses; stack allocation reclaims the message on frame unwind with no allocator or collector involvement. Removing the mailbox operations removes their synchronization traffic on the memory bus; quantifying that reduction requires measurement and is left to future work. The chained-execution result — $O(1)$ rather than $O(N)$ context switches across an N -actor synchronous chain — is the largest structural effect for deep call graphs.

Table 3. Per-message operations on the asynchronous path and their status on the synchronous path, for co-located actors. This is an analytical account of work removed.

Per-message operation	Asynchronous	Synchronous
Message heap allocations	1–3	0 request (stack-alloc.), 1 reply
Mailbox queue operations	2 (enqueue + dequeue)	0
Scheduler invocations	1–2	0
Context switches (request/reply)	2–4	0
Context switches, N -actor chain	$O(N)$	$O(1)$

Context: measured cost of the asynchronous path. To indicate the magnitude of the overhead the synchronous path targets, the asynchronous messaging path of the same actor runtime has been measured, in separate work [17], at 5.5 to 13.6 million round trips per second after successive batching and memory-pooling optimizations; in the saturated throughput regime a message costs on the order of 74 ns, while in the sparse regime an individual send costs several microseconds (4–7 μ s), with condition-variable wakeups costing 1–3 μ s each. These are measurements of the *asynchronous* path and bound the per-message costs that synchronous delivery removes; they are not measurements of `fast_send`. We turn now to direct microbenchmarks of the synchronous path itself.

11.1 Microbenchmarks of the C++ implementation

Table 3 predicts that eliminating each per-message operation produces a measurable reduction in latency. The framework’s C++ microbenchmark suite measures these steps on a *sequential* round trip — ping $\rightarrow$ pong $\rightarrow$ reply with a single message in flight, so each sample is a latency rather than a saturated throughput [40]. The figures below were taken on an AMD EPYC 9374F (RHEL 9, g++ 15), compiled `-O3 -march=native` and pinned with `taskset` to two cores, reported as the median of three runs. Absolute values depend on hardware, allocator, thread placement, and turbo state; the durable result is the *ratio* between rows, each of which isolates one eliminated operation. The `steady_clock` resolution on this machine quantizes to ~ 10 ns, so a per-sample `fast_send` percentile is resolution-bound; `fast_send` costs are therefore also reported as amortized ns (total wall time over N operations, a single clock-read pair), which resolves costs below one tick.

Table 4 isolates the cost of crossing threads; the two paragraphs that follow it decompose the internal overhead of `fast_send` and the effect of the memory pool.

The cost is thread-crossing, not messaging. Table 4 holds the work constant — one ping/pong round trip with a reply — and varies only the delivery mechanism, so each row adds back one operation. A cross-thread asynchronous `send` between two actors on separate cores costs $\approx 3.4 \mu$ s at the median. This cost is dominated by the two cross-core mailbox wakeups per round trip, each a mutex acquisition and a condition-variable signal on which the receiving thread must be scheduled; it is placement-sensitive, because the cross-core path depends on which cores the two threads occupy. Keeping both threads but letting each busy-poll its mailbox instead of sleeping removes the wakeups and nothing else: the round trip falls to 370 ns. Of the $\sim 3.4 \mu$ s, therefore, some 3 μ s is putting threads to sleep and waking them, and about 370 ns — two crossings of ~ 185 ns — is moving the message and the queue slot between cores; the price of spinning is a core per actor, busy whether or not a message arrives. Co-scheduling the two actors on one thread (grouping) removes the crossings as well and reduces the round trip to 90 ns, a $37\times$ reduction. Replacing the remaining mailbox push/pop with an inline `fast_send` reduces it to ~ 30 ns (amortized; the

per-sample $p50$ is resolution-bound), because a `fast_send` is mechanically a locked function call: it takes the receiver’s lock and runs the handler inline. What that costs against the dispatch a programmer would otherwise write by hand is measured separately, in Table 5.

Table 4. The cost is thread-crossing, not messaging. Same ping/pong round trip with a reply; only the delivery mechanism varies. Each row eliminates one operation relative to the row above. “Wakeup” is a cross-core mailbox wakeup of a sleeping thread (mutex + condition variable); “queue” is a mailbox push/pop. The $p50$ column is per-sample round-trip latency; the `fast_send` figure is amortized, because its per-sample $p50$ is resolution-bound (the `steady_clock` tick is ~ 10 ns), and its speedup is computed on amortized figures. The grouped and `fast_send` rows show negligible spread. In the spinning row both actors keep their own threads on separate cores, but each thread busy-polls its mailbox (the lock-free mailbox with a consumer that never sleeps), so a message still crosses cores but never waits for a thread to wake. Three runs: $p50$ 370–380 ns (median 370), $p99$ 400–410 ns.

Delivery mode	Threads	Wakeup	Queue	$p50$ RT (ns)	Speedup
<code>send</code> , ungrouped	2	yes	yes	~ 3370	$1\times$
<code>send</code> , ungrouped, spinning	2	no	yes	370	$9.1\times$
<code>send</code> , grouped	1	no	yes	90	$37\times$
<code>fast_send</code> (inline)	1	no	no	~ 30	$\sim 110\times$

Against hand-written dispatch. Table 4 compares actor delivery paths with each other. The comparison that decides whether to use the framework at all is against the call a programmer would otherwise write. Table 5 measures that on the same machine: every arm does the same work on a stack-allocated message and differs only in how the work is reached. It measures dispatch alone, without a reply, so its `fast_send` figure is below the round trip of Table 4. Direct and virtual calls are real, non-inlined calls. A virtual call costs very little when the branch predictor knows its target and several nanoseconds when it does not, so the table reports both: a single “virtual call costs X ” figure would depend entirely on how predictable the call site is.

Table 5. `fast_send` against hand-written dispatch. Same work in every arm; amortized ns per call, minimum of 15 runs of 5×10^6 calls, AMD EPYC 9374F, RHEL 9.2, `g++ -O3 -march=native`, pinned to two cores. Dispatch only, no reply. Source: `actors/cpp/perf/bench_dispatch.cpp`.

Dispatch	ns	$\times$ direct
direct call (not inlined)	1.63	1.0
virtual call, one target (predicted)	1.86	1.1
virtual call, 2–8 targets (mispredicted)	7.53–9.32	4.6–5.7
<code>fast_send</code>	9.10	5.6

`fast_send` costs about one virtual call whose target the predictor cannot guess, and about $5\times$ a predicted virtual call or a direct call: some 7.5 ns more per hop, for the receiver’s lock, message-field setup, handler lookup and reply plumbing. Whether that is small depends on what it is compared with. Against the $\sim 7\ \mu\text{s}$ socket-to-book floor of Section 12 it is under 1%; on a leaner path — a hardware-timestamped, wire-to-book pipeline measured in hundreds of nanoseconds — the same 7.5 ns per hop would matter.

Grouping, and why `fast_send` improves on it. The grouped row is the strongest conventional alternative. The framework places a set of actors in a **Group**, co-scheduled on a single thread behind

a single mailbox, so an asynchronous `send` within the group never crosses a core: it is a push and a pop on a queue owned by the running thread. This is the standard low-latency actor optimization — same-thread messaging in Actor4j, a shared dispatcher in Akka (Section 3) — and it reduces the round trip from $\sim 3.4\ \mu\text{s}$ to 90 ns. Grouping still pays for the queue: the message is enqueued, the run loop dequeues it, and the handler runs on a later turn. `fast_send` removes that layer. On the same thread it does not enqueue; it takes the receiver’s lock and runs the handler inline, returning the reply as a value, in ~ 30 ns — about $3\times$ faster than grouping. Grouping removes the thread hop; `fast_send` removes the thread hop *and* the queue. This synchronous, inline, receiver-transparent delivery is, to our knowledge, absent from the mainstream actor systems surveyed in Section 3. The intended use follows: group the actors of a servicing stage onto one thread and drive them with `fast_send`, so that an N -actor pipeline collapses to a single inline call chain (the middle section of Figure 2).

Allocation, not dispatch, is the only variable cost. Beyond the inline dispatch, the only per-message cost a caller pays is for memory it chooses to allocate. A request message kept on the stack requires none, because `fast_send` blocks until the handler returns (Section 6). For a message that must outlive the call, allocation from a `MemoryPool` replaces a general-purpose `new/delete`. On a grouped-`send` round trip with two allocations, the pool reduces the amortized cost from ~ 128 ns (pool off) to ~ 92 ns (pool on). The larger and platform-independent benefit is the tail: in a separate run the maximum round trip fell from 5.5 ms to $33\ \mu\text{s}$ once pooled, because the pool never takes the general allocator’s occasional slow path. This is the quantity that matters for Section 10: a two-order-of-magnitude reduction of the $p99.9+$ tail that a slow servicing stage feeds into a growing queue.

Scope. These are sequential latencies with one message in flight, not saturated throughput, and they do not vary contention or chain depth. The established results are: `fast_send` costs tens of nanoseconds; its overhead over a direct call is ~ 7.5 ns plus whatever memory the caller allocates; it improves on same-thread grouping by about $3\times$; and pooling removes the latency tail. The multi-axis, saturated evaluation (Section 14) remains future work.

11.2 Mailbox queue types and their selection

The mailbox is a `std::variant` over four multi-producer/single-consumer queue implementations, selectable per actor: `BQueue` (a mutex and condition variable around a ring buffer); `BQueueBatched` (the same lock, but the consumer drains the whole queue under one acquisition); `ShardedBQueue` (N independent lanes, each separately locked, joined by an atomic cursor); and `LockFreeMPSC` (a bounded lock-free Vyukov ring, in which a producer reserves a slot with a single atomic operation). Because the variant holds the concrete type, `push` and the drain loop are devirtualized and inlined. Table 6 measures the four on the EPYC target across the three regimes that separate them; a fourth regime, a single cross-thread message, distinguishes none of them, because thread placement moves the figure by $2.2\times$ — more than the gap between queues.

No single queue wins everywhere. On a shared (grouped) thread, `LockFreeMPSC` has the lowest median (70 ns, $1.3\times$ below `BQueue`), because its push and pop are each an uncontended atomic operation with no lock or wakeup. Under a bursty single producer — the market-data regime of the case study — `BQueueBatched` is fastest (584.9 ns/msg): draining an entire packet burst under one lock acquisition amortizes the lock over the burst, and `BQueue` trails by under 4%. Under 32-producer fan-in, `ShardedBQueue` wins decisively — $p99$ of $6.6\ \mu\text{s}$ against `BQueue`’s $69.9\ \mu\text{s}$ ($10.6\times$), and $3.6\times$ the throughput — because its independent lanes remove the single-mutex serialization

Table 6. Four mailbox queue types, one column per distinguishing regime (EPYC target, pinned, median of three runs). *Grouped*: single-thread round-trip median. *Burst*: single bursty producer, amortized cost per message. *Fan-in*: 32 concurrent producers, push *p99*. Bold marks the best in each regime; no type is best in all three.

Queue type	Grouped <i>p50</i> (ns)	Burst (ns/msg)	Fan-in <i>p99</i> (μ s)
BQueue	90	605.8	69.9
BQueueBatched	90	584.9	72.4
ShardedBQueue	190	983.1	6.6
LockFreeMPSC	70	672.0	16.5

that dominates the other three; it pays for those lanes in the other regimes, where it is the slowest of the four.

The selection rule follows. Default every actor to **BQueue**: it is fastest or within a few percent in every regime except fan-in, and it has the most stable tail of the four (grouped $\max$ 2.4–9 μ s, versus 14–16 μ s for **LockFreeMPSC**). Override to **ShardedBQueue** only for the few actors written to by many producers concurrently — a shared order-out actor, or a fan-in aggregator across market-data handlers — the one regime in which it wins. **LockFreeMPSC**, despite the best grouped median, is not recommended as a default because its tail is the least stable measured; **ShardedBQueue** is not a default because its lanes are wasted, and counterproductive, outside fan-in.

12 Case study: tick-to-book latency in a live market-data path

The microbenchmarks of Section 11.1 measure the framework’s per-message cost in isolation, and the queueing analysis of Section 10 predicts, in principle, where latency comes from. This section grounds both in a live measurement of the KASPAR-HFT market-data path on production exchange data, and asks the paper’s title question directly: is the actor model’s overhead small enough for a high-frequency market-data path? The measurements below are drawn from a companion latency study of the same system [41].

12.1 Setup

The path under measurement is the market-data ingest of Section 10.2: read a UDP datagram off CME MDP 3.0 multicast, decode the Simple Binary Encoding (SBE) messages [42] inside it, apply them to an order book, and publish. Two timestamps are taken per message — t_0 when the datagram comes up from the socket, t_1 when the book update is published — and their difference is recorded for every message. Structurally this is two actor stages: a socket-reader actor hands each datagram to a decode-and-order-book actor by an asynchronous **send** (necessarily asynchronous — a blocking reader would stop draining its socket and drop packets), and decode plus book construction run together on the second thread. The handoff between the two stages is the decoder’s mailbox — a bounded ring buffer of datagrams (a **BQueue**) — and we write *qlen* for its occupancy: the number of datagrams already waiting in the ring when a given datagram is handed to the decoder, so *qlen* = 0 means the ring was empty and the datagram was picked up at once. This is the “ring” and the “queue” referred to throughout this section. The handoff is a single bursty producer, the regime in which **BQueueBatched** is marginally faster than the **BQueue** used here (Section 11.2); the difference is a few percent of the mailbox cost and negligible against the ~ 7 μ s floor. Three instruments (E-mini S&P 500, ES; E-mini Nasdaq-100, NQ; 10-year Treasury Note, ZN), each with a book-update and a trade stream: 8.29 million messages over 53 minutes of a busy afternoon (2026-09-16). One caveat

Table 7. The per-message model median $\approx \text{floor} + \text{slope} \times \text{idx}$, fitted on medians at queue length zero. The floor (a message first in its packet) agrees within $0.42 \mu\text{s}$ across six streams; the slope (the in-packet serialization cost) varies $\sim 3\times$ and is where the tail is bought.

Stream	Floor (μs)	Slope (ns/msg)
ES book	6.98	566
NQ book	7.23	312
ZN book	6.83	966
ES trade	6.81	714
NQ trade	7.14	526
ZN trade	6.81	965

bounds everything below: t_0 is a *software* timestamp taken at the socket read, so time spent in the NIC and the kernel is excluded. This is socket-to-book, not wire-to-book.

12.2 The per-message model: floor plus serialization

One UDP datagram carries many SBE messages, decoded in order. Write idx for a message’s position within its packet, counted from zero: $\text{idx} = 0$ is the message decoded first, $\text{idx} = 1$ the next, and so on. A message at position idx therefore waits for the idx decodes ahead of it before its own runs — a wait that sits inside its measured latency, and that is zero exactly when $\text{idx} = 0$. Restricting to messages that arrive to an empty ring (nothing queued ahead of their packet), so the only remaining wait is in-packet, the median latency is linear in idx ,

$$\text{median latency} \approx \text{floor} + \text{slope} \times \text{idx},$$

with the two coefficients of Table 7, fitted on medians. (Means are useless here: a single multi-millisecond stall in a cell moves the mean by tens of microseconds, so every quantity in this section is a median or a percentile.) The two coefficients carry the argument.

The *floor* — the intercept, a message first in its packet — agrees to within $0.42 \mu\text{s}$ across six independent streams, $6.8\text{--}7.2 \mu\text{s}$. It is the irreducible cost of decoding one SBE message and applying it to the book; it is insensitive to queue length; and it is stable under load, moving by at most $+0.4 \mu\text{s}$ (and *falling* on two of three instruments) during a Federal Reserve release that raised the packet rate $4.6\text{--}9.8\times$ in a second. Against this $\sim 7 \mu\text{s}$ floor the framework’s per-message contribution — the mailbox handoff and dispatch of Section 11.1 — is tens of nanoseconds: a **fast_send** hop adds $\sim 7.5 \text{ ns}$ over a direct call (about one mispredicted virtual call, Table 5; the full ping/pong round trip is $\sim 30 \text{ ns}$). *The actor abstraction is under 1% of the floor*; the floor is SBE decode and book mutation, not message delivery.

The *slope* — the per-message serialization cost, $0.31\text{--}0.97 \mu\text{s}$ — determines the tail. In-packet position scales linearly into latency: the last message of a 45-message packet incurs $44 \times 0.57 \approx 25 \mu\text{s}$ on top of the floor, deterministically. On the trade streams a message is essentially never alone (99.3–99.6% have another message ahead of them in the same packet), which is why the trade medians already sit above the book medians. Shaving 100 ns off the floor moves every message by 100 ns; shaving it off the slope moves a message by $\text{idx} \times 100 \text{ ns}$, and idx is largest in the tail. The design lever is therefore the slope rather than the floor or the mailbox queue, which is rare: between 87% and 99.7% of messages arrive to find the ring empty.

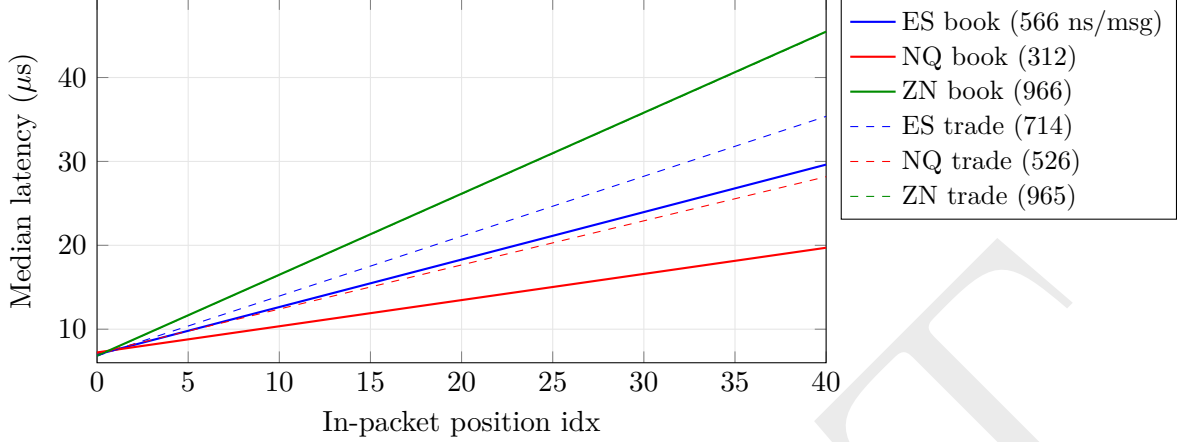


Figure 3. The per-message model median = floor + slope $\times$ idx (Table 7), one line per stream. All six share a $\sim 7 \mu s$ floor at idx = 0; the slope — the per-message serialization cost — sets the divergence, from 312 ns/msg (NQ book) to 966 ns/msg (ZN book).

12.3 Three estimates of the floor

The floor of Table 7 is a fitted intercept. Two further estimates, constructed differently, agree with it. The first isolates the hot path directly: the subset of messages with qlen = 0 and idx = 0, for which neither the queue nor in-packet serialization contributes, so its median is the floor with no model interposed (Table 8). Across six streams the spread is $1.41 \mu s$; across the three book streams it is $0.35 \mu s$.

Table 8. Direct hot-path floor: median latency of the subset at qlen = 0 and idx = 0, with the sample size. No queue and no in-packet serialization contribute to these messages.

Stream	Median (μs)	n
ES book	7.01	1,781,767
NQ book	7.24	1,841,447
ZN book	6.89	671,271
ES trade	7.62	1,008
NQ trade	8.30	298
ZN trade	7.96	474

The second estimate is the fitted idx = 0 intercept of the median ladder (Table 7). The third is measured during the Federal Open Market Committee statement, when the packet rate rose $6.5\times$, $4.6\times$, and $9.8\times$ on the three instruments within a second; taking the floor from single-message packets (span ≈ 1) in that window gives the third row of Table 9. The three agree to within a few tenths of a microsecond.

Table 9. Three independent estimates of the floor (μs), book streams. The three methods differ in their sample, their estimator, and the load regime they were taken under, and agree to within $0.4 \mu s$.

Method	ES	NQ	ZN
Direct median (qlen = 0, idx = 0)	7.01	7.24	6.89
Fitted idx = 0 intercept (median ladder)	6.98	7.23	6.83
Floor at span ≈ 1 during the FOMC release	7.1	7.5	7.0

Table 10. Unconditional socket-to-book latency distribution (μ s), every admitted message per stream, no conditioning. The median is uniform across streams; the tail is not.

Stream	Messages	$p50$	$p90$	$p99$	$p99.9$	max
ES book	2,861,519	7.1	10.6	18.5	41.2	1148.4
NQ book	3,792,033	7.3	9.9	13.6	24.7	5567.0
ZN book	1,119,746	7.1	12.1	57.0	180.9	2458.2
ES trade	296,736	9.0	15.3	38.0	127.7	421.5
NQ trade	110,003	8.3	12.5	31.2	83.7	694.2
ZN trade	114,154	15.4	69.3	219.4	409.5	504.8

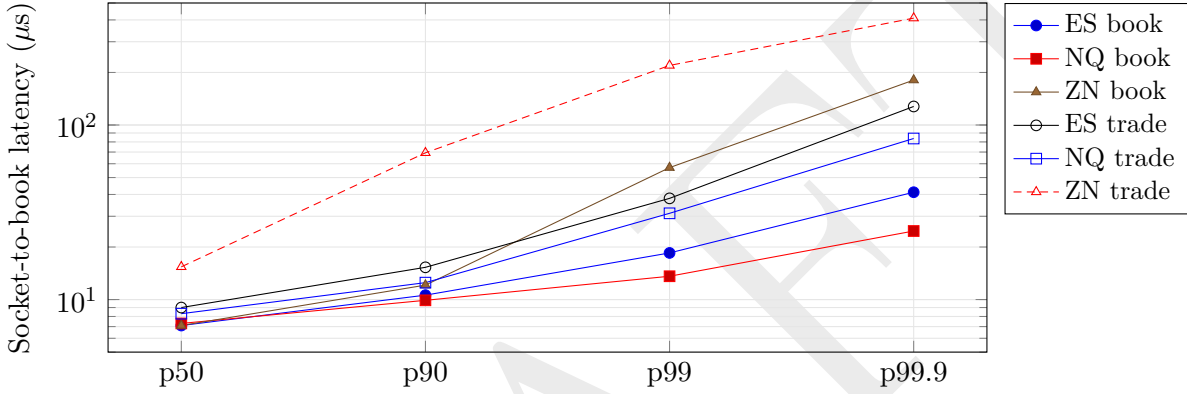


Figure 4. Socket-to-book latency percentiles per stream (Table 10), log scale. Medians cluster near 7–15 μ s; the tails diverge, ZN trade reaching 219 μ s at $p99$ and 410 μ s at $p99.9$.

The third estimate is the informative one. During the release the floor moved by at most +0.4 μ s, and fell on two of the three instruments. The hot path is not rate-sensitive: three methods, three instruments, all within 6.8–7.5 μ s. This is the most stable quantity in the study — it moved negligibly when the sample tripled, when the estimator changed from mean to median, and when the packet rate rose tenfold.

12.4 The distribution and its tail

A trading operation is judged on the tail, not the median. Table 10 gives the unconditional distribution. The medians cluster — three separate multicast channels, three instruments, all within 0.2 μ s — but the tails come apart: ES book’s $p99$ is 18.5 μ s, ZN book’s is 57.0 μ s, and ZN trade’s is 219 μ s. Expressed as amplification from median to $p99$, the streams range from $1.9\times$ (NQ book) to $14.2\times$ (ZN trade). A single “8 μ s” figure for this system would be true and would say nothing about the quantity that matters.

The tail is set by the in-packet serialization of Section 12.2, not by the framework: a message’s latency is floor plus slope times its position, and a bursty feed produces long packets, so the tail is that slope multiplied by a large position.

12.5 Isolating the queue from packet position

The floor-plus-slope model of Section 12.2 and the distribution of Table 10 both aggregate over queue occupancy. Grouping latency by qlen and reporting a within-group statistic measures the queue only if nothing else varies across the groups. Something does: a burst that fills the ring tends

also to produce larger packets, so a message at high qlen also sits, on some streams, deeper inside its own packet. Part of the rise a naive qlen table attributes to the queue is therefore in-packet position. The correction is the move of Section 12.2 applied again: hold $\text{idx} = 0$, so the message is first in its packet and no in-packet serialization remains in its latency, and vary qlen. Whatever still moves is the queue.

Estimator. Within a qlen cell the *mean* is not a measure of the queue. A cell may hold tens of messages, and a single multi-millisecond stall moves the mean by tens of microseconds: on one cut NQ book’s mean at $\text{qlen} = 0$ was $280\ \mu\text{s}$ against a median of $7.4\ \mu\text{s}$. Stalls are a separate phenomenon; every figure in this section is a median or a percentile.

Columns. Tables 11–14 carry eleven columns. *qlen* is the number of packets already in the ring when this message’s packet was handed to the consumer. *msgs* is the count at that queue length — the denominator for the pooled quantiles, and it collapses quickly (ES book has 2.66 million messages at $\text{qlen} = 0$ and 51 at $\text{qlen} = 6$). *med*, *p90*, *p99* are the latency quantiles pooled over in-packet position; these are the confounded columns and form an upper bound on the queue’s cost. *span* is the mean packet size (SBE messages per datagram) of those messages — the suspected confounder — and *idx* their mean in-packet position, which is approximately $\text{span}/2$. n_0 is the subset first in their own packet, the deconfounded sample; med_0 , p90_0 , p99_0 are the quantiles of that subset, with the bracketed figure in med_0 the change from the $\text{qlen} = 0$ row. Percentiles are suppressed (–) where n_0 is below roughly 200, at which point a 99th percentile is the largest sample rather than a measurement.

Table 11. ESZ6 book: queue occupancy versus latency, deconfounded from packet position. Latencies in μs . Pooled quantiles (*med*, *p90*, *p99*) run over all in-packet positions; *span*/*idx* are the mean packet size and in-packet position; n_0 and the $_0$ quantiles are the subset first in their packet ($\text{idx} = 0$). Bracketed: change in med_0 from $\text{qlen} = 0$.

qlen	msgs	med	p90	p99	span	idx	n_0	med_0	p90_0	p99_0
0	2,664,628	7.1	10.4	17.9	2.20	0.62	2,303,948	6.9	9.8	13.9
1	181,719	6.5	12.3	24.2	1.78	0.39	162,136	6.3 (–0.6)	11.4	17.9
2	12,790	7.2	12.8	42.2	1.45	0.21	12,175	7.1 (+0.2)	12.3	22.2
3	1,619	8.6	16.5	44.6	1.20	0.06	1,547	8.5 (+1.5)	15.7	35.0
4	376	10.2	92.8	309.7	4.17	1.57	320	9.8 (+2.8)	21.8	79.2
5	134	15.3	348.6	–	9.64	4.26	97	11.5 (+4.6)	27.5	–
6	51	17.0	–	–	1.18	0.08	48	16.6 (+9.7)	–	–
7	90	398.0	445.2	–	26.00	12.50	22	n/a	–	–
8	41	459.7	–	–	14.46	6.73	18	n/a	–	–

The queue is convex. Reading med_0 down each book stream isolates the queue with position held at zero. ES: 6.9, 6.3, 7.1, 8.5, 9.8, 11.5, 16.6 μs , successive increments $-0.6, +0.8, +1.4, +1.3, +1.7, +5.1$. NQ: increments $-1.3, +1.0, +1.4, +2.9, +5.2$. ZN: $-0.9, +1.6, +1.8, +2.2, +10.1$. Three independent multicast channels give three convex curves with the confounder held fixed — the shape the M/M/1 model of Section 10 predicts.

Table 12. NQZ6 book: queue occupancy versus latency, deconfounded. Columns as in Table 11.

qlen	msgs	med	p90	p99	span	idx	n_0	med ₀	p90 ₀	p99 ₀
0	3,310,403	7.4	9.9	13.5	2.40	0.80	2,586,119	7.2	9.5	12.4
1	463,440	6.1	9.0	13.6	2.11	0.58	371,010	5.9 (−1.3)	8.6	12.9
2	15,274	7.0	11.6	25.8	1.92	0.36	13,508	6.9 (−0.3)	11.4	20.5
3	1,641	8.6	17.8	168.5	2.82	0.79	1,457	8.3 (+1.1)	14.3	34.3
4	340	12.1	91.8	456.4	3.39	1.17	276	11.2 (+4.0)	27.6	456.4
5	121	17.7	384.8	—	1.39	0.21	109	16.4 (+9.2)	382.0	—
6	72	16.8	393.5	—	1.19	0.12	64	16.8 (+9.5)	414.5	—
7	57	19.4	—	—	1.51	0.35	46	22.8 (+15.5)	—	—
8	36	95.4	—	—	2.06	0.50	29	n/a	—	—
9	42	74.0	—	—	1.95	0.55	34	67.7 (+60.5)	—	—

Table 13. ZNZ6 book: queue occupancy versus latency, deconfounded. Columns as in Table 11. ZN is the stream whose packets grow with queue length; compare med and med₀.

qlen	msgs	med	p90	p99	span	idx	n_0	med ₀	p90 ₀	p99 ₀
0	988,149	7.2	11.9	43.9	3.13	1.12	829,350	6.9	10.2	15.4
1	124,649	6.3	12.6	88.8	3.41	1.22	105,162	6.0 (−0.9)	9.4	17.0
2	4,831	10.8	97.3	266.6	12.02	5.49	2,740	7.6 (+0.7)	14.2	72.8
3	1,088	84.9	214.0	307.4	18.83	8.92	430	9.4 (+2.5)	27.3	160.3
4	477	128.2	228.5	458.5	23.33	11.18	130	11.6 (+4.7)	69.4	—
5	206	256.0	326.8	334.5	26.50	12.74	44	21.7 (+14.8)	—	—
6	70	163.3	191.5	—	17.21	8.11	31	18.3 (+11.4)	—	—
7	115	193.4	342.0	—	26.65	12.83	21	n/a	—	—
8	50	150.2	—	—	20.56	9.78	12	n/a	—	—

Table 14. Trade streams: queue occupancy versus latency, deconfounded. Columns as in Table 11, with a leading stream label. The trade streams reach qlen ≥ 1 on too few messages for the deconfounded subset to support percentiles.

stream	qlen	msgs	med	p90	p99	span	idx	n_0	med ₀	p90 ₀	p99 ₀
ES	0	295,337	9.0	15.2	34.6	8.51	4.25	1,522	7.9	11.2	15.7
ES	1	1,242	10.3	128.7	202.7	22.48	11.41	14	n/a	—	—
ES	2	129	144.7	216.8	—	57.64	28.36	2	n/a	—	—
NQ	0	109,541	8.3	12.5	29.6	7.06	3.63	416	8.2	10.7	32.7
NQ	1	387	8.5	74.7	93.9	16.07	7.91	5	n/a	—	—
NQ	2	65	100.7	139.7	—	83.72	41.58	0	n/a	—	—
ZN	0	108,706	15.0	61.0	142.4	32.03	16.00	649	8.4	62.9	111.4
ZN	1	4,213	50.5	191.2	285.9	30.68	15.28	107	76.8 (+68.4)	170.5	—
ZN	2	755	219.5	385.0	415.2	71.95	36.29	9	n/a	—	—
ZN	3	139	234.8	259.5	—	68.78	39.65	2	n/a	—	—

The convexity is concentrated in the tail. The queue’s effect on the median is small; its effect on the upper percentiles is roughly an order of magnitude larger. Table 15 gives the ratio from $qlen = 0$ to $qlen = 3$ at each quantile. With position held at zero, three queued packets cost ES $+1.5 \mu s$ at the median but $+21 \mu s$ at $p99$, and ZN $+145 \mu s$ at $p99$. The pooled columns — every message at that queue length, whatever its packet position, the figure an operator experiences — are larger again, and diverge from the deconfounded ratios exactly where packets grow with queue length: on ES the two agree (ES packets do not grow with $qlen$), on ZN they do not.

Table 15. Queue cost from $qlen = 0$ to $qlen = 3$, as a ratio at each quantile. Left: position held at $idx = 0$ (queue only). Right: pooled over in-packet position (queue + packet, as experienced). The two agree on ES, where packets do not grow with queue length, and diverge on ZN, where they do.

Stream	At $idx = 0$			Pooled over idx		
	$p50$	$p90$	$p99$	$p50$	$p90$	$p99$
ES book	$1.23\times$	$1.60\times$	$2.52\times$	$1.21\times$	$1.59\times$	$2.49\times$
NQ book	$1.15\times$	$1.51\times$	$2.77\times$	$1.16\times$	$1.80\times$	$12.48\times$
ZN book	$1.36\times$	$2.68\times$	$10.41\times$	$11.79\times$	$17.98\times$	$7.00\times$

Frequency. The effect is confined to a rare event. ES book reaches $qlen \geq 3$ on 2,311 of 2,861,519 messages, 0.08%. The queue is a large effect at the percentiles on a small fraction of messages, not a small effect everywhere.

Packet growth splits the streams. The *span* column separates two regimes. On ZN, packets grow as the ring fills — ZN book runs $3.13 \rightarrow 3.41 \rightarrow 12.02 \rightarrow 18.83 \rightarrow 23.33 \rightarrow 26.50$ messages per packet from $qlen = 0$ to 5 — so a message at high $qlen$ is both behind more packets and deeper inside a larger one, and the naive table charges both to the queue. Table 16 quantifies the overstatement: ZN book’s raw median rise at $qlen = 3$ is $+77.7 \mu s$ but $+2.5 \mu s$ at $idx = 0$, an overstatement of $31\times$. On ES and NQ book, packets *shrink* as the ring fills (ES $2.20 \rightarrow 1.20$, NQ $2.40 \rightarrow 1.19$ over the rows carrying weight), there is no upward confounding to remove, and med_0 tracks the pooled median (ES $+0.2/ +1.5$ deconfounded against $+0.1/ +1.5$ pooled). Two caveats: ES span is not monotone — it falls to 1.20 at $qlen = 3$ then rises to 4.17 and 9.64 at $qlen = 4, 5$, on 376 and 134 messages, a handful of bursts rather than a regime — and NQ at $qlen = 3$ shows span 2.82 against 2.40, a mild rise. The supported statement is that ES and NQ do not exhibit ZN’s eightfold growth, not that their span falls monotonically.

Table 16. ZNZ6 book: the packet-growth confounder, quantified. Raw rise is the pooled-median change from $qlen = 0$; the $idx = 0$ column is the same change with packet position held at zero; the ratio is the overstatement in the naive column. Rises in μs .

$qlen$	raw rise	at $idx = 0$	overstated by
1	-0.9	-0.9	(both negative)
2	+3.6	+0.7	$5.1\times$
3	+77.7	+2.5	$31\times$
4	+121.0	+4.7	$26\times$
5	+248.8	+14.8	$17\times$

The mechanism is a reading of the pattern, not a separate measurement: ES and NQ book run fast with a median packet of one message, so their ring backs up with many small packets and being

queued indicates a high arrival rate rather than a large packet; ZN is quiet, and the main event that fills its ring is a genuine burst, which enlarges packets at the same time.

Result. The queue has the shape queueing theory predicts — convex on all three book streams — on a rare event ($qlen \geq 3$ at 0.08% of messages), with a magnitude that depends on the quantile: about $+1.5 \mu s$ at the median and $+21$ to $+145 \mu s$ at $p99$. Where packets grow with queue length (ZN), a naive $qlen$ table overstates the median rise by up to $31\times$; the deconfounded columns remove that. This does not alter the suitability conclusion below: the queue’s contribution, like the floor’s and the slope’s, is a property of the feed and the servicing time, not of the actor framework, whose ~ 10 ns hop is below every figure in these tables.

12.6 Suitability

The framework does not appear in this decomposition. The floor is decode and book work, and the slope is protocol serialization; neither is the framework. The actor abstraction on the collapsed chain — one `fast_send` dispatch per hop — costs about one mispredicted virtual call per hop (~ 9 ns), a few tens of nanoseconds across the collapsed decode/book/signal chain, against a $\sim 7 \mu s$ floor: well under 1%, and it does not grow with in-packet position, so it makes no measurable contribution to the tail under this workload.

This also places the queueing analysis of Section 10 correctly. That analysis predicts the tail diverges as utilization $\rho \rightarrow 1$, and it is the right theory for a *saturated* servicing stage. This stage is not saturated: ρ is far from 1, since almost every message finds an empty ring, so queueing is not the operative tail mechanism here — the in-packet serialization of a bursty, self-excited feed is. And ρ itself is governed by the decode-and-book service time ($\sim 7 \mu s$), of which the messaging framework is under 1%; it is the per-message work, not the delivery mechanism, that determines whether this stage saturates.

The verdict is affirmative and specific: the actor model is suitable for a high-frequency market-data path because its per-message overhead is a fraction of a percent of the irreducible work and makes no measurable contribution to the observed tail. What an implementer must engineer is the per-message service time — the decode-and-book slope — and the feed’s batch structure, not the messaging framework.

12.7 Design implications

The measurements yield a specific design rule. In $med = floor + slope \times idx$ the two terms do different jobs: the floor sets the median, the slope sets the tail. Across the six streams the floor varies by $0.42 \mu s$ (6.81 to 7.23) and the slope by $3.1\times$; the $p99$ column of Table 10 is bought almost entirely with the slope. This gives a test for whether an optimization is worth doing: removing 100 ns from the floor moves every message by 100 ns, while removing 100 ns from the slope moves a message by $idx \times 100$ ns, and idx is what is large in the tail. On ZN book the $p99$ message sits at position 25, where 100 ns of slope is worth $2.5 \mu s$ against $0.1 \mu s$ at the median.

Where messages sit. The packet-size distribution is packet-weighted, the wrong weighting for this question, because a larger packet holds more messages and therefore more of the latency. Table 17 weights by message. On the books, four messages in five arrive alone in their packet ($idx = 0$) and pay the floor only — the median idx is zero on all three. On the trades, 99.3% to 99.6% of messages have another ahead of them; a trade message almost always shares its packet,

and the median ZN trade message is ninth in its packet, having incurred eight slopes before it is decoded.

Table 17. In-packet position, message-weighted. Distribution of idx over all admitted messages per stream. On the books the median message is first in its packet; on the trades it is not.

Stream	Messages	idx = 0	mean	p50	p90	p99	max
ES book	2,861,519	86.7%	0.60	0	1	11	44
NQ book	3,792,033	78.4%	0.77	0	3	10	41
ZN book	1,119,746	83.8%	1.17	0	2	25	44
ES trade	296,736	0.5%	4.29	2	9	33	84
NQ trade	110,003	0.4%	3.67	1	9	32	84
ZN trade	114,154	0.7%	16.21	8	47	77	84

Position reproduces the medians. Evaluating the fitted ladder at each stream’s own position quantiles reproduces the observed medians with no appeal to queueing (Table 18). ZN trade’s median is 15.4 μ s against ES book’s 7.1 μ s, and 14.5 of those 15.4 follow from being ninth in the packet on the stream with the largest slope — two multiplied factors, neither a queue. This is a check rather than an identity: the median of a sum is not the sum of medians, and composing an idx quantile with a median-fitted ladder is looser still. The prediction is low on all six rows, by 0.05 to 0.83 μ s at the median, with the error growing in idx; the right-hand block is a lower bound on the tail, not a forecast.

Table 18. Position reproduces the medians. The fitted floor-plus-slope ladder evaluated at each stream’s own idx quantiles (i@p50, i@p99), against observed latency. *cover* is the predicted p99 as a fraction of the observed p99: how much of the tail in-packet position accounts for.

Stream	floor	slope	i@p50	pred p50	obs p50	i@p99	pred p99	obs p99	cover
ES book	6.98 μ s	566 ns	0	6.98 μ s	7.08 μ s	11	13.2 μ s	18.5 μ s	71%
NQ book	7.23 μ s	312 ns	0	7.23 μ s	7.28 μ s	10	10.4 μ s	13.6 μ s	76%
ZN book	6.83 μ s	966 ns	0	6.83 μ s	7.10 μ s	25	31.0 μ s	57.0 μ s	54%
ES trade	6.81 μ s	714 ns	2	8.24 μ s	9.02 μ s	33	30.4 μ s	38.0 μ s	80%
NQ trade	7.14 μ s	526 ns	1	7.67 μ s	8.34 μ s	32	24.0 μ s	31.2 μ s	77%
ZN trade	6.81 μ s	965 ns	8	14.53 μ s	15.36 μ s	77	81.1 μ s	219.4 μ s	37%

Coverage is the actionable column. Where the ladder accounts for 70–80% of the p99, position is the whole story and the slope is worth attacking; where it accounts for 37% (ZN trade), something else owns the other 63%. The same split appears in the time budget (Table 19), which is sum-weighted and therefore fully exposed to stalls — the purpose of its residual column. The ladder is fitted at qlen = 0 on medians, so the residual is queueing plus stalls plus anything else it does not capture. On NQ book the residual is 1.0%: the model is the system. On ZN trade it is 26.3%, which no tuning of the decode loop will reach. The order of work follows: measure the residual first; if it is small, optimize the slope; if it is large, the slope is not the problem and the cause must be found — in this system, a stall population that remains unexplained.

Three approaches that do not help. *Reducing the message rate.* The ranking runs the other way: NQ book is the busiest stream (806 packets/s) and has the best p99 (13.6 μ s); ZN book is the

Table 19. Time budget, sum-weighted. Total decode-and-book time per stream and its split into floor, slope, and residual (queueing, stalls, and anything else the $qlen = 0$ median ladder does not capture). Being sum-weighted, this is the one table fully exposed to stalls.

Stream	Total (ms)	Floor	Slope	Residual
NQ book	28,624	95.8%	3.2%	1.0%
ES book	21,851	91.4%	4.5%	4.1%
NQ trade	1,037	75.7%	20.5%	3.8%
ES trade	3,147	64.2%	28.9%	6.9%
ZN book	10,429	73.3%	12.1%	14.6%
ZN trade	3,477	22.4%	51.3%	26.3%

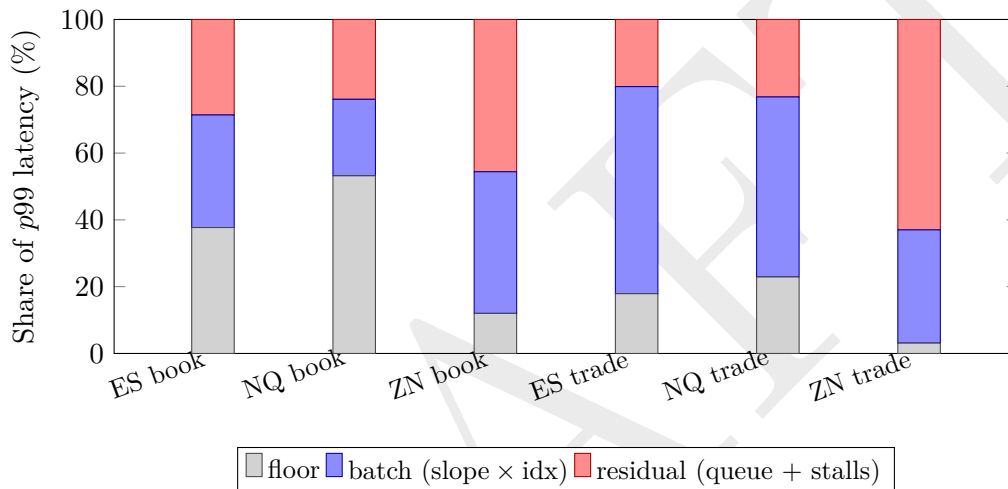


Figure 5. The tail is not the floor. Composition of each stream’s $p99$ socket-to-book latency: the floor, the in-packet batch ($slope \times idx$ evaluated at the $p99$ position), and the residual (queue and stalls); absolute $p99$ ranges from 13.6 to 219.4 μs (Table 18). The floor — decode, book, and the actor framework — is a minority of the tail and falls to 3–12% on the worst streams (ZN); the structured growth is the batch term. The queue, which fires on 0.08% of messages, is only a small part of the residual; the remainder is the stall population.

quietest (241 packets/s) and the worst (57.0 μs). During the FOMC release ZN book grew faster — $12.4 \rightarrow 7.7 \mu s$ — at ten times the packet rate. A stage with ρ far from 1 does not behave like one near saturation, and this one is not: 87% to 99.7% of messages arrive to an empty ring. *Sizing the ring.* Those same fractions show most messages never queue; added ring capacity changes nothing at the median or at the $p99$ of the book streams. The queue is real — $qlen \geq 3$ costs +1.5 μs at $p50$ and +21 to +145 μs at $p99$ — but it fires on 0.08% of messages. *Batching harder.* The slope is charged per message regardless of arrival pattern; coalescing does not reduce the work, it moves messages to higher idx , the variable that lengthens tails. The trade streams are what a batched book stream looks like: the same floor, 0.4% of messages at $idx = 0$, and a $p99$ two to four times worse.

An open question on the slope. The $3.1\times$ spread between NQ and ZN is reproducible and unexplained. If it is cache residency, then making the cold path cheaper means making it smaller — harder than shaving instructions. If it is message mix, ZN messages are simply more expensive and there may be nothing to win. Resolving this requires hardware performance counters and is left

open.

12.8 The arrival process

The sections above reduce the tail to packet span, and span is set by how many messages arrive together — a property of the arrival process. The natural null is a Poisson process, and it fails in two independent ways. A Poisson process requires both that the marginal gaps be exponentially distributed and that successive gaps be independent; both fail here, for different reasons and with different consequences.

Background: what Poisson means and how to reject it. A *Poisson process* is the standard model of “random, independent” arrivals and the assumption under which the queueing results of Section 10 hold: the time between successive arrivals is exponentially distributed, and each gap is independent of the last, so arrivals neither cluster nor anticipate one another [36]. If the market-data feed were Poisson, packets would arrive one at a time and evenly spread, the queue would rarely build, and the tail would follow from utilization alone. It is not, and four standard statistics measure how far it departs. The *coefficient of variation* (CV) is the standard deviation of the gaps over their mean; it is exactly 1 for a Poisson process and larger when the gaps are more uneven than exponential. The *Fano factor* [47] is the variance of the number of arrivals in a fixed window divided by the mean of that count; it is 1 for a Poisson process at every window size, rises above 1 when arrivals cluster, and keeps rising *with the window* when the clustering persists across timescales. The *Hurst exponent* H [48] summarizes that scaling: $H = 0.5$ for a Poisson or any independent process, whereas $H > 0.5$ signals long-range dependence — bursts nested within bursts rather than one characteristic clump size. To decide whether a departure is due to the *sizes* of the gaps or their *order*, we apply a *Fisher–Yates shuffle* [51] — the standard algorithm that puts a list into a uniformly random order (the computing equivalent of a fair shuffle of a deck of cards, with every one of the $n!$ orderings equally likely). Applied to the sequence of gaps it keeps the exact multiset of gap sizes unchanged and destroys only their order, so any statistic that falls after shuffling was measuring order, not size. Finally, a *Hawkes process* [49, 50] is a self-exciting point process in which each arrival transiently raises the rate of further arrivals — a model of contagion or momentum — whose *branching ratio* is the mean number of further arrivals each arrival directly triggers; a value near 1 is “near-critical” and produces large, long bursts. Figure 6 gives the intuition; the rest of this section makes it quantitative.



Figure 6. Why the feed is not Poisson (schematic). Each row contains the same number of packet arrivals over the same interval. A Poisson process of that average rate spreads them out (bottom); the measured feed clusters them into bursts separated by idle gaps (top). The clustering is what fills the ring and enlarges packets. It is quantified — not invented — in Figure 7 and Tables 20–23, where the measured process reaches a Fano factor above 1000 against Poisson’s 1. The tick positions here are illustrative, not the measured timestamps.

One sampling note: the two tests below read raw packet arrival timestamps from a separate side file (arrival times only, no latency), over the whole capture after the startup replay rather than the 14:25–15:18 latency window. The sample n is therefore larger than in the latency tables and the rates differ slightly; the Fano statistic at a 5-second window needs the full span.

The marginal is not exponential. The fair null is an exponential with the same mean, which fixes the average rate so that every difference is shape rather than level; for $\text{Exp}(1/m)$ the p -th quantile is $-m \ln(1 - p)$. Table 20 compares the two on ES book. The ratio is below 1 through $p90$ and above 1 in the tail, crossing once: most packets arrive far closer together than a Poisson of the same rate would place them, paid for by rare long idles — the signature of a burst process.

Table 20. ESZ6 book interarrival gaps against an exponential of the same mean ($n = 4,154,158$, mean gap $1793.6 \mu\text{s}$, rate $557.5/\text{s}$). The ratio crosses 1 once.

Quantile	Measured (μs)	Exp, same mean (μs)	Ratio
$p10$	7.22	188.97	$0.04\times$
$p50$	74.60	1243.22	$0.06\times$
$p90$	3196.83	4129.88	$0.77\times$
$p99$	32182.96	8259.77	$3.90\times$
$p99.9$	119057.18	12389.65	$9.61\times$

The quantity that matters for a ring buffer is the share of packets that arrive nearly on top of the one before. For any exponential this is a constant, $P(\text{gap} < \text{mean}/10) = 1 - e^{-0.1} = 9.52\%$. Table 21 shows it reaching 85% on ZN book: these are the back-to-back runs that fill the ring and enlarge the packet, and a Poisson source at the identical average rate would essentially never deliver them. Both CV and CV^2 are tabulated with their definitions, since the two are frequently confused.

Table 21. Interarrival-gap dispersion. CV is the coefficient of variation of the gaps and CV^2 its square. $P(\text{gap} < \text{mean}/10)$ is the share of near-back-to-back arrivals; for any exponential it is 9.52%.

Stream	CV	CV^2	$P(\text{gap} < \text{mean}/10)$	vs Poisson 9.52%
ES book	10.97	120.4	73.94%	$7.8\times$
NQ book	28.97	839.1	63.11%	$6.6\times$
ZN book	13.60	184.9	84.82%	$8.9\times$
ES trade	4.10	16.8	56.01%	$5.9\times$
NQ trade	4.78	22.8	39.97%	$4.2\times$
ZN trade	6.11	37.4	84.20%	$8.8\times$

Equivalently, the decoder never experiences the average rate. Table 22 contrasts the mean packet rate with the instantaneous rate implied by the median gap; for an exponential that ratio is $1/\ln 2 = 1.44\times$ at any rate. ZNbook is quoted at 241 packets per second, yet the typical back-to-back pair arrives at an instantaneous rate near twenty-nine thousand.

Table 22. The decoder does not experience the average rate. Mean packet rate against the instantaneous rate implied by the median gap. For an exponential the ratio is $1.44\times$.

Stream	Mean rate	Rate from median gap	Ratio
ES book	557.5/s	13,405/s	$24.1\times$
NQ book	806.0/s	11,555/s	$14.3\times$
ZN book	240.9/s	29,028/s	$120.5\times$

The order is not independent either. A non-exponential marginal alone would still admit a renewal process — heavy-tailed gaps drawn independently. The control is a seeded Fisher–Yates

shuffle of the gap sequence, which preserves the marginal exactly and destroys only the ordering; if the burstiness lived in the marginal, the shuffle would change nothing. Table 23 reports the result. Gap autocorrelation is positive at all seven lags tested on all six streams (42 of 42 outside the white-noise band): long gaps follow long, short follow short. The Fano factor — variance of the count in a window over its mean, 1.0 for Poisson at every window size — reaches 1024–1260 on the book streams at a 5-second window and collapses by $2.8\times$ to $18.5\times$ under the shuffle. That collapse is the result: the burstiness lives in the ordering, because the shuffle left the marginal untouched and still removed most of it. Fano scaling gives $\text{Fano}(T) \sim T^{2H-1}$; the measured Hurst exponent is 0.643–0.743 against a shuffled null of 0.600–0.660 and 0.5 for any renewal process. If the generating process is Hawkes, the Fano ceiling implies a branching ratio $n \leq 1 - 1/\sqrt{\text{Fano}}$ of 0.850 to 0.972 — each arrival triggering close to one further arrival, a system just below criticality.

Table 23. Ordering statistics. Gap ACF L1 is the lag-1 gap autocorrelation; Fano(5s) is the count Fano factor at a 5-second window (1.0 for Poisson at any window); *shuffled* recomputes it after a seeded Fisher–Yates shuffle of the gap sequence, which preserves the marginal and destroys the order; *collapse* is their ratio. H is the Hurst exponent from Fano scaling (0.5 for Poisson or any renewal process); H shuffled is the same on the shuffled series.

Stream	gap ACF L1	Fano(5s)	shuffled	collapse	H	H shuffled
ES book	+0.040	1024.2	55.3	$18.5\times$	0.743	0.622
NQ book	+0.005	1260.0	103.1	$12.2\times$	0.702	0.660
ZN book	+0.052	1133.5	69.4	$16.3\times$	0.711	0.624
ES trade	+0.131	65.6	12.8	$5.1\times$	0.680	0.604
NQ trade	+0.065	44.3	7.1	$6.2\times$	0.695	0.600
ZN trade	+0.284	73.0	25.7	$2.8\times$	0.643	0.615

Two caveats. The shuffle is not a control for a slowly drifting rate, which shuffling also destroys; this sample cannot separate self-excitation from non-stationarity, and both read as clustering. And the shuffled null is 0.60–0.66, not 0.5 — finite-sample bias in the Hurst estimator at these window counts — so the effect is the gap between 0.74 and 0.62, not the distance from 0.5.

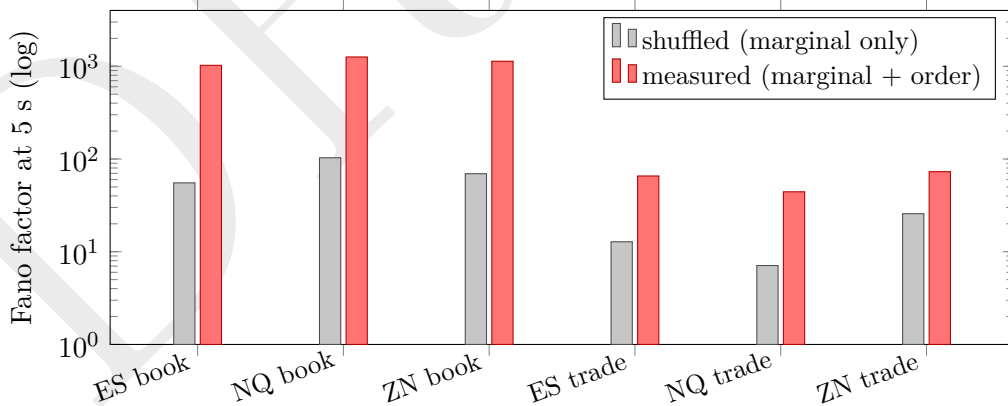


Figure 7. Two reasons the arrivals beat Poisson. Count Fano factor at a 5-second window (Table 23); Poisson is 1 at every window — the baseline at the bottom of the axis. The shuffled bar, with the gap sequence reordered and the marginal preserved, is the burstiness the marginal alone carries; the measured bar adds the ordering. Both tower over the Poisson baseline, and the measured/shuffled gap (2.8 – $18.5\times$) is contributed by correlation that a marginal statistic such as c_a^2 cannot see.

Why this lengthens the tail beyond Poisson. The mechanism is mechanical. Clustered arrivals place several packets inside one decode interval; CME coalesces what it can into a single datagram, so clustering converts into span — more SBE messages per UDP packet — and span converts into latency linearly, message k paying $k \times \text{slope}$ at 0.31–0.97 μs per message with no randomness. A Poisson source at 557 packets/s would leave 1.79 ms between packets and deliver a span-1 packet almost every time; the measured stream has a 74 μs median gap and span-45 packets, and the last message of one pays $44 \times 0.57 = 25 \mu\text{s}$ of serialization on top of a 7 μs floor. The tail originates in the arrival process: clustering becomes packet span, and span becomes latency through the slope.

This is also why the standard queueing correction is insufficient. Kingman’s approximation [46] $\mathbb{E}[W] \approx \frac{\rho}{1-\rho} \cdot \frac{c_a^2 + c_s^2}{2} \cdot \mathbb{E}[S]$ admits burstiness only through c_a^2 , the squared CV of the marginal, and understates here in two independent ways. First, the shuffle shows that correlation contributes beyond anything the marginal can explain, and c_a^2 cannot see ordering. Second, ρ is far from 1 — 87% to 99.7% of messages find the ring empty — so the $\rho/(1-\rho)$ term does almost no work. The queue term is small and the variability term is blind to the mechanism that actually produces the tail. The queue in front of the packet is convex, as Section 12.5 shows, fires on 0.08% of messages, and when it fires costs +1.5 μs at the median and +21 to +145 μs at $p99$: a rare, sharp effect on top of a common, smooth one. The batch accounts for the bulk of the tail mass; the queue accounts for why the far tail exceeds what the batch alone predicts.

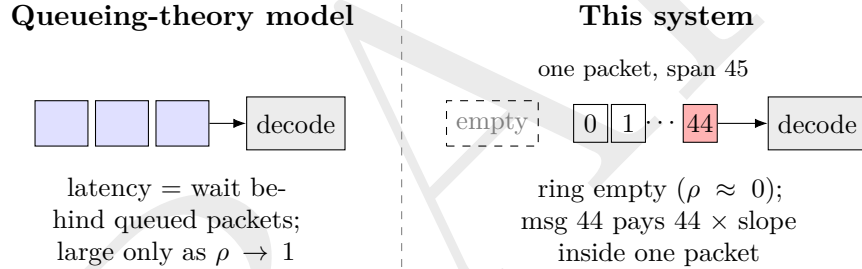


Figure 8. Why queueing theory misses the tail. The M/M/1 model charges latency to packets waiting in the mailbox, which is large only near saturation. In the measured system the ring is empty 87–99.7% of the time ($\rho \approx 0$), yet one coalesced packet can carry dozens of messages, and the k -th pays $k \times \text{slope}$ of serialization before it is decoded — a tail the queueing model cannot see, because it assumes one service per arrival.

12.9 Limits of this measurement

t_0 is a software timestamp at the socket read, so this is socket-to-book, not wire-to-book; hardware receive timestamps would close that gap and were not taken. The data is one box and one 53-minute session; a 12-minute cut of the same session gave per-message slopes that moved by up to $2.4\times$, so this window is not obviously sufficient either. The per-message service cost varies $\sim 3\times$ across instruments (ZN is dearest), and whether that is cold-cache residency or a genuinely heavier message mix cannot be separated without hardware counters. The deep-queue rows ($\text{qlen} \geq 5$) are too sparse to characterize their tail, so the queue result rests on queue lengths 0–3 on the three book streams. And a separate population of multi-millisecond stalls — the largest latencies in the dataset — is real and remains unexplained here.

13 Production and simulation: a duality of the group

The group defines a single message order. A Group co-schedules its member actors on one thread behind one shared mailbox (Section 11.1): the members do not own separate queues; the group’s single queue holds every message addressed to any member, and the run loop dispatches them one at a time. Beyond the latency effect measured earlier — the collapse of the round trip from $\sim 3.4\ \mu\text{s}$ to 90 ns — this has a structural consequence. A single queue imposes a single total order on all messages entering the group, so the group’s entire execution — every actor state transition and every outgoing message — is a deterministic function of that one input sequence.

The same actors run in production and in simulation. That property makes one group two systems. In production the queue is fed by live sources — socket readers, timers, market-data actors — and the run loop executes in real time. In simulation the queue is fed from a recorded or synthetic sequence, and the run loop executes as fast as the host permits. The member actors are identical in both modes: the framework gives a handler nothing that distinguishes a message from a live socket from one read from a backtest file, which is the receiver transparency of Section 7 extended from the delivery mechanism to the data source. As there, this is a property of the interface, not isolation the language enforces. No strategy actor is edited, recompiled, or conditionally branched to move between live trading and backtest, and replaying the same input sequence reproduces the same states and outputs. The one discipline the duality imposes on an actor is that it acquire time, randomness, and external input through messages or injected interfaces rather than through global calls, so that the simulator can supply them deterministically.

Relation to prior work. Two ingredients of this design have prior art; their combination is uncommon. The single-shared-queue group is a first-class facility in Actor4j [11], where a group’s actors run on one thread with their mailboxes outsourced to the thread’s queue — presented there as a message-passing throughput optimization, not as a simulation mechanism. The same-code production/simulation duality is the principle of deterministic simulation testing: FoundationDB’s Flow runtime [44] and systems such as TigerBeetle run unmodified production code inside a single-threaded deterministic simulator, but there the unit of determinism is an asynchronous continuation or an explicit state machine rather than a group of mailbox actors sharing a queue. Agent-based market simulators such as ABIDES [45] drive actor-like agents behind one time-ordered event queue, but those agents exist only in simulation and are not a deployed production system. The design here couples the two: the shared-queue group is the seam that makes mode-agnostic actors run identically in real-time production and in deterministic backtest. We are not aware of this specific combination being described elsewhere.

Example: developing an execution algorithm. This duality is how the order-level shadowing execution algorithm of the companion paper [43] was developed. The strategy is an actor that consumes market-data and order-event messages and emits child-order messages; it was built and validated entirely in simulation, driven from recorded CME sessions, and then deployed to production without modification — the same actor and the same handlers, reading from a live feed instead of a file. The 246-session slippage comparison reported there was produced by the simulation mode of the same group that runs the strategy live. The child-order logic is written once and is agnostic to the mode in which it executes.

14 Discussion and limitations

The scope is co-located actors. `fast_send` applies only when sender and receiver share an address space; a synchronous cross-machine call would reintroduce the network round trip it exists to avoid, and the actor model’s location transparency means the sender can fall back to asynchronous delivery when the receiver is remote. Receiver transparency is what makes that fallback free: the same handler serves the remote asynchronous case and the local synchronous case.

The cost of transparency. Heap-allocating the reply in both modes is a deliberate cost paid to keep the handler unable to distinguish them. The optimized synchronous-only reply removes that allocation but forfeits transparency and must therefore be confined to code paths statically known to be synchronous; mixing it with asynchronous delivery is an error the mechanism reports rather than tolerates.

Blocking runs counter to conventional guidance. Established actor frameworks discourage blocking inside actors precisely because it can starve thread pools and cause deadlock [7, 6, 10]. `fast_send` blocks the sender by construction, and its safety rests on the cyclic-invocation detector and on the availability of the non-blocking and best-effort variants for paths that cannot tolerate an unbounded wait. Where a caller must never block, `fast_send_x` and `fast_send_void` provide bounded-time and never-blocking alternatives behind the same handler interface. Even so, the call-chain detector addresses only cycles on one thread (Section 8), and only *cyclic* waits; avoiding thread-pool starvation for acyclic blocking patterns — a chain that blocks on an actor whose handler is itself waiting on a scarce resource — remains a burden on the developer that asynchronous delivery does not impose.

Contention on a hot actor. A synchronous send holds the receiver’s exclusive-access lock for the duration of handler execution, so concurrent synchronous sends to the same heavily contended actor serialize on that lock, and the blocking variants stall their caller threads while they wait. Under heavy contention this can perform worse than asynchronous delivery, in which senders enqueue and continue and the messages buffer without stalling the callers. The non-blocking (`fast_send_x`) and best-effort (`fast_send_void`) variants bound or remove the stall — the latter by falling back to the mailbox — so the ratio of synchronous completions to asynchronous fallbacks doubles as a signal that a target has become hot enough that the asynchronous path is preferable.

Permissive detection shifts a burden to the programmer. The (actor, message-class) refinement admits callbacks by allowing an actor’s state to be re-entered by a nested handler, which reintroduces the reentrancy hazards that strict per-actor sequential processing removes. A nested handler can observe and mutate the actor’s state mid-update, which relaxes the actor model’s guarantee of single-threaded sequential state mutation and admits the invariant violations that guarantee is designed to prevent. The conservative policy avoids this at the cost of rejecting some legitimate callback structures; the choice between them is exposed as configuration rather than fixed.

Further evaluation. The microbenchmarks of Section 11.1 establish the per-operation latency reductions and the two-order-of-magnitude tail benefit of the memory pool directly. The natural next step measures the mechanism under load: synchronous versus asynchronous *throughput* across contention levels, chain depths, and message sizes on the x86-64 Linux target, together with the

distribution of synchronous completions versus asynchronous fallbacks for `fast_send_void`. That saturated, multi-axis characterization is the subject of ongoing work.

15 Future work

1. **Detecting cycles across threads** (Section 8), which a thread-local call chain cannot see: a global wait-for graph, a global lock order, or a bounded try-lock with asynchronous fallback.
2. **The saturated-throughput and contention evaluation** (Section 14).
3. **Completing kaspar-hft**: the cyclic-invocation detector and its refinement, the three remaining delivery operations, the optimized synchronous-only reply, and enforcement of a single reply per message.

16 Conclusion

The actor model has long been dismissed for high-frequency trading on a plausible but mistaken premise: that its safety is inseparable from a discipline — one thread per actor, a mailbox per actor, a message and a context switch for every interaction — whose per-message cost cannot fit a microsecond budget. The premise conflates the model with one way of implementing it. For *co-located* actors the discipline can be honored without that cost, and the resulting system is fast enough for the HFT hot path while keeping the properties that made the model attractive in the first place: per-actor state isolation, freedom from data races, and sequential single-message reasoning. The synchronous path gives up part of the asynchronous model’s freedom from deadlock: a cycle of synchronous calls spread across threads deadlocks. This paper supports that thesis both analytically and with a deployed, measured implementation — *kaspar-hft* — rather than as theory alone.

Asynchronous message passing is intrinsic to the actor model’s guarantees and indispensable across machines, but for co-located actors it imposes heap allocation, queue operations, scheduling, and context switches that isolation does not require. `fast_send` provides a synchronous alternative in which the sender’s thread runs the receiver’s handler directly under the receiver’s exclusive-access lock and takes the reply as a value, while the handler’s code does not depend on the delivery mode, the executing thread, or the sender’s blocked state — receiver transparency. A thread-local call-chain membership test, checked before lock acquisition, catches synchronous cycles on one thread before they deadlock or overflow the stack; cycles across threads need a different mechanism. A refinement keyed on (actor, message-class) pairs admits legitimate callbacks while still catching unbounded recursion, and five delivery operations expose the latency/robustness spectrum behind one transparent handler interface. The performance case is made analytically, as the elimination of per-message work, and corroborated by microbenchmarks in which the synchronous path completes a round trip in tens of nanoseconds and a memory pool cuts the latency tail by two orders of magnitude.

The applied study places these numbers against a live market-data path. On the *kaspar-hft* framework, socket-to-book latency decomposes into a $\sim 7 \mu\text{s}$ decode-and-book floor and a per-message serialization slope; the floor is stable to within a few tenths of a microsecond across three estimators and a tenfold change in packet rate, and the framework’s own per-hop cost ($\sim 9 \text{ ns}$, about one unpredictable virtual call) is well under 1% of it. The tail is not the framework and not, primarily, the queue: with in-packet position held fixed the mailbox queue is convex exactly as queueing theory predicts but fires on only 0.08% of messages, while the dominant effect is the arrival process, which is non-Poisson and strongly clustered, consistent with a self-exciting process (under a

Hawkes interpretation the Fano scaling implies a branching ratio $\approx 0.85\text{--}0.97$), and converts bursts into large packets and in-packet position into latency through the slope. The design lever is therefore the per-message service time, not the queue depth or the message rate. The same study also fixes the choice among the four mailbox queue implementations — BQueue by default, ShardedBQueue for high-fan-in actors — and shows that a group of actors sharing one queue is at once a real-time execution engine and a deterministic simulator, so a trading strategy is developed in backtest and deployed to production without modification.

Taken together, the mechanism and the measurement answer the question in the title. The four extensions — `fast_send`, actor groups, per-actor mailbox selection, and the memory pool — reduce the actor framework’s contribution on a live tick-to-book path to under one percent of the irreducible decode-and-book floor, and to no measurable contribution to the tail, which belongs to the market’s arrival process and not to the messaging layer. An HFT data path built on this model therefore pays essentially no latency penalty for the actor discipline, and in return obtains data-race freedom, single-message reasoning that is unusually amenable to automatic code generation, and a production/simulation duality in which the same strategy code is backtested and traded without change. The actor model, long held unsuitable for high-frequency trading, is suitable, and a real system trades on it. The remaining work is listed in Section 15.

References

- [1] C. Hewitt, P. Bishop, and R. Steiger. *A Universal Modular ACTOR Formalism for Artificial Intelligence*. Proc. 3rd International Joint Conference on Artificial Intelligence (IJCAI), 1973.
- [2] G. Agha. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, 1986.
- [3] M. S. Miller, E. D. Tribble, and J. Shapiro. *Concurrency Among Strangers: Programming in E as Plan Coordination*. Symposium on Trustworthy Global Computing, 2005.
- [4] T. Van Cutsem et al. *AmbientTalk: Object-Oriented Event-Driven Programming in Mobile Ad Hoc Networks*. XXVI Int. Conf. of the Chilean Computer Science Society, 2007.
- [5] J. Armstrong, R. Virding, C. Wikström, and M. Williams. *Concurrent Programming in Erlang*. Prentice Hall, 2nd ed., 1996.
- [6] J. Armstrong. *Making Reliable Distributed Systems in the Presence of Software Errors*. PhD thesis, Royal Institute of Technology (KTH), Stockholm, 2003.
- [7] Lightbend, Inc. *Akka Documentation*. <https://akka.io/docs/>.
- [8] D. Charousset, T. C. Schmidt, R. Hiesgen, and M. Wählich. *CAF — The C++ Actor Framework for Scalable and Resource-Efficient Applications*. Proc. 4th Int. Workshop on Programming based on Actors, Agents, and Decentralized Control (AGERE!), 2014.
- [9] S. Bykov et al. *Orleans: Cloud Computing for Everyone*. Proc. 2nd ACM Symposium on Cloud Computing (SoCC), 2011.
- [10] Apple, Inc. *Swift Evolution Proposal SE-0306: Actors*. 2021.
- [11] D. Bauer. *Actor4j: A Software Framework for the Actor Model Focusing on the Optimization of Message Passing*. 2018.

- [12] O. Tardieu et al. *Reliable Actors with Retry Orchestration*. IBM Research, 2021.
- [13] R. G. Lavender and D. C. Schmidt. *Active Object: An Object Behavioral Pattern for Concurrent Programming*. Pattern Languages of Program Design 2, Addison-Wesley, 1996.
- [14] D. C. Schmidt, M. Stal, H. Rohnert, and F. Buschmann. *Pattern-Oriented Software Architecture, Volume 2: Patterns for Concurrent and Networked Objects*. Wiley, 2000.
- [15] H. Sutter and J. Larus. *Software and the Concurrency Revolution*. ACM Queue, 3(7):54–62, 2005.
- [16] M. Lohstroh, C. Menard, S. Bateni, and E. A. Lee. *High-Performance Deterministic Concurrency using Lingua Franca*. arXiv:2301.02444, 2023.
- [17] V. Maciejewski. *How Message Batching More Than Doubles Actor-Model Throughput*. <https://vincentmayeski.substack.com/p/how-message-batching-more-than-doubles>, 2026.
- [18] I. G. Greif. *Semantics of Communicating Parallel Processes*. PhD thesis, MIT, 1975. Project MAC Technical Report MAC-TR-154.
- [19] C. Hewitt and H. Baker. *Laws for Communicating Parallel Processes*. In *Information Processing 77* (Proc. IFIP Congress 77), North-Holland, 1977, pp. 987–992.
- [20] W. D. Clinger. *Foundations of Actor Semantics*. PhD thesis, MIT, 1981. MIT AI Lab Technical Report AI-TR-633.
- [21] J. Armstrong. *A History of Erlang*. Proc. 3rd ACM SIGPLAN Conference on the History of Programming Languages (HOPL III), 2007. <https://doi.org/10.1145/1238844.1238850>.
- [22] C. Hewitt. *Actor Model of Computation: Scalable Robust Information Systems*. arXiv:1008.1459, 2010 (rev. 2015). <https://arxiv.org/abs/1008.1459>.
- [23] E. Stenman. *The Erlang Runtime System (The BEAM Book)*, chapter “Scheduling”. <https://github.com/happi/theBeamBook>, accessed 2026.
- [24] Lightbend (Akka Team). *50 Million Messages per Second — on a Single Machine*. <https://letitcrash.com/post/20397701710/50-million-messages-per-second-on-a-single>, 2012.
- [25] S. Bykov, A. Geller, G. Kliot, J. R. Larus, R. Pandya, and J. Thelin. *Orleans: Distributed Virtual Actors for Programmability and Scalability*. Microsoft Research Technical Report MSR-TR-2014-41, 2014.
- [26] S. Clebsch, J. Franco, S. Drossopoulou, A. M. Yang, T. Wrigstad, and J. Vitek. *Orca: GC and Type System Co-design for Actor Languages*. Proc. ACM Program. Lang. 1(OOPSLA), Article 72, 2017.
- [27] C. Camilleri, J. G. Vella, and V. Nezval. *Actor Model Frameworks: An Empirical Performance Analysis*. In *Advances in Information and Communication* (FICC 2023), Lecture Notes in Networks and Systems, vol. 652, Springer, 2023. https://doi.org/10.1007/978-3-031-31153-6_37.

- [28] S. M. Imam and V. Sarkar. *Savina — An Actor Benchmark Suite: Enabling Empirical Evaluation of Actor Libraries*. Proc. 4th Int. Workshop on Programming based on Actors, Agents, and Decentralized Control (AGERE!), 2014. <https://doi.org/10.1145/2687357.2687368>.
- [29] S. Blessing, K. Fernandez-Reyes, A. M. Yang, S. Drossopoulou, and T. Wrigstad. *Run, Actor, Run: Towards Cross-Actor Language Benchmarking*. Proc. 9th Int. Workshop on Programming based on Actors, Agents, and Decentralized Control (AGERE!), 2019.
- [30] D. Charousset, T. C. Schmidt, R. Hiesgen, and M. Wählich. *Native Actors — A Scalable Software Platform for Distributed, Heterogeneous Environments*. arXiv:1301.0748, 2013.
- [31] Actix Project. *Actix: Actor Framework for Rust — Documentation*. <https://docs.rs/actix/>, accessed 2026.
- [32] S. Tasharofi, P. Dinges, and R. E. Johnson. *Why Do Scala Developers Mix the Actor Model with Other Concurrency Models?* Proc. 27th European Conference on Object-Oriented Programming (ECOOP), LNCS 7920, Springer, 2013, pp. 302–326.
- [33] M. Bagherzadeh, N. Fireman, A. Shawesh, and R. Khatchadourian. *Actor Concurrency Bugs: A Comprehensive Study on Symptoms, Root Causes, API Usages, and Differences*. Proc. ACM Program. Lang. 4(OOPSLA), Article 214, 2020. <https://doi.org/10.1145/3428282>.
- [34] C. Torres Lopez, S. Marr, H. Mössenböck, and E. Gonzalez Boix. *A Study of Concurrency Bugs and Advanced Development Support for Actor-based Programs*. In *Programming with Actors*, LNCS 10789, Springer, 2018, pp. 155–185. Also arXiv:1706.07372.
- [35] J. Huang, T. Mahmud, C. Păsăreanu, and G. Yang. *CONCUR: Benchmarking LLMs for Concurrent Code Generation*. arXiv:2603.03683, 2026.
- [36] L. Kleinrock. *Queueing Systems, Volume 1: Theory*. Wiley-Interscience, 1975.
- [37] C. Li, C. Ding, and K. Shen. *Quantifying the Cost of Context Switch*. Proc. Workshop on Experimental Computer Science (ExpCS), ACM, 2007.
- [38] L. McVoy and C. Staelin. *lmbench: Portable Tools for Performance Analysis*. Proc. USENIX Annual Technical Conference, 1996.
- [39] E. Bendersky. *Measuring Context Switching and Memory Overheads for Linux Threads*. <https://eli.thegreenplace.net/2018/measuring-context-switching-and-memory-overheads-for-linux-threads/>, 2018.
- [40] V. Maciejewski. *kaspar-hft actor framework: C++ message-passing microbenchmarks (actors/cpp/perf)*. <https://github.com/vincent212/kaspar-hft>, 2026.
- [41] V. Maciejewski. *The Queue Is Inside the Packet: Market-Data Latency Measured One Message at a Time*. kaspar-hft technical report (tech_reports/md_latency_article.md), 2026. <https://github.com/vincent212/kaspar-hft>.
- [42] FIX Trading Community. *Simple Binary Encoding (SBE), Version 1.0*. Technical Standard, 2020. <https://www.fixtrading.org/standards/sbe/>.
- [43] V. Maciejewski. *Model-Free Passive Execution via Order-Level Shadowing*. kaspar-hft technical report (tech_reports/shadow_pov), 2026. <https://github.com/vincent212/kaspar-hft>.

- [44] J. Zhou, M. Xu, A. Shraer, et al. *FoundationDB: A Distributed Unbundled Transactional Key Value Store*. In Proc. ACM SIGMOD, 2021. <https://doi.org/10.1145/3448016.3457559>.
- [45] D. Byrd, M. Hybinette, and T. H. Balch. *ABIDES: Towards High-Fidelity Multi-Agent Market Simulation*. In Proc. ACM SIGSIM-PADS, 2020. <https://doi.org/10.1145/3384441.3395986>.
- [46] J. F. C. Kingman. *The Single Server Queue in Heavy Traffic*. Mathematical Proceedings of the Cambridge Philosophical Society, 57(4):902–904, 1961. <https://doi.org/10.1017/S0305004100036094>.
- [47] U. Fano. *Ionization Yield of Radiations. II. The Fluctuations of the Number of Ions*. Physical Review, 72(1):26–29, 1947. <https://doi.org/10.1103/PhysRev.72.26>.
- [48] H. E. Hurst. *Long-Term Storage Capacity of Reservoirs*. Transactions of the American Society of Civil Engineers, 116:770–799, 1951. <https://doi.org/10.1061/TACEAT.0006518>.
- [49] A. G. Hawkes. *Spectra of Some Self-Exciting and Mutually Exciting Point Processes*. Biometrika, 58(1):83–90, 1971. <https://doi.org/10.1093/biomet/58.1.83>.
- [50] E. Bacry, I. Mastromatteo, and J.-F. Muzy. *Hawkes Processes in Finance*. Market Microstructure and Liquidity, 1(1):1550005, 2015. <https://doi.org/10.1142/S2382626615500057>.
- [51] R. A. Fisher and F. Yates. *Statistical Tables for Biological, Agricultural and Medical Research*. Oliver & Boyd, London, 1938. (Shuffle algorithm; see also D. E. Knuth, *The Art of Computer Programming*, Vol. 2, Algorithm P.)